

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGUYỄN PHẠM CHÍ BẢO

LEVA – HỆ THỐNG TRỢ LÝ ẢO CỤC BỘ

ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH

TP. HỒ CHÍ MINH, 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGUYỄN PHẠM CHÍ BẢO

LEVA – HỆ THỐNG TRỢ LÝ ẢO CỤC BỘ

Mã số sinh viên: 2351010017

ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH

Giảng viên hướng dẫn: TS.Lê Phú Nguyên

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án em đã nhận được sự hướng dẫn, hỗ trợ và động viên quý báu từ quý thầy cô. Những sự giúp đỡ này là nguồn động lực quan trọng để em có thể hoàn thành đề tài cũng như tích lũy thêm nhiều kiến thức và kinh nghiệm thực tế.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến thầy Lê Phú Nguyên, giảng viên hướng dẫn, người đã tận tình hướng dẫn, định hướng và đưa ra những góp ý quý báu trong suốt quá trình thực hiện đồ án. Những kiến thức, nhận xét và chỉ dẫn của thầy đã giúp em định hình rõ hơn hướng thực hiện đồng thời khắc phục những khó khăn trong quá trình nghiên cứu và từng bước hoàn thiện đề tài.

Em cũng xin chân thành cảm ơn quý thầy cô Trường Đại học Mở Thành phố Hồ Chí Minh đặc biệt là các thầy cô trong ngành Công nghệ Thông tin vì đã tận tâm giảng dạy và trang bị cho em những kiến thức nền tảng cũng như chuyên môn cần thiết trong suốt quá trình học tập. Đây là cơ sở quan trọng để em có thể vận dụng kiến thức vào quá trình nghiên cứu và xây dựng đồ án hiện tại.

Em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

TÓM TẮT ĐỒ ÁN NGÀNH

Đồ án trình bày quá trình nghiên cứu, thiết kế và triển khai hệ thống trợ lý ảo hoạt động chủ yếu trong mạng cục bộ, hướng đến việc giảm sự phụ thuộc vào các dịch vụ trí tuệ nhân tạo trên nền tảng đám mây và xây dựng một giải pháp có chi phí vận hành thấp, phù hợp với phần cứng cá nhân và các thiết bị IoT.

Hệ thống được xây dựng có bốn thành phần chính bao gồm:

- _ Voice Node sử dụng vi điều khiển ESP32-S3 kết hợp với micro kỹ thuật số INMP441 để thu âm và bộ khuếch đại I2S MAX98357A để phát âm thanh.

- _ Weather Node sử dụng ESP8266 kết hợp cảm biến BMP280 nhằm thu thập dữ liệu môi trường và cung cấp thông tin nhiệt độ và áp suất cho hệ thống.

- _ Local AI Server được triển khai trên máy tính cá nhân và đóng vai trò xử lý trung tâm, sử dụng FastAPI để xây dựng giao tiếp với các Node IoT nối tiếp Faster-Whisper được dùng để chuyển đổi giọng nói thành văn bản sau đó Qwen2.5 được triển khai cục bộ thông qua Ollama để xử lý ngôn ngữ và tạo câu trả lời và cuối cùng dùng edge-tts và pydub để tổng hợp văn bản thành giọng nói.

- _ Blynk được sử dụng để xây dựng giao diện giám sát dữ liệu và trạng thái hệ thống từ xa.

Quy trình hoạt động của hệ thống bắt đầu khi người dùng phát lệnh bằng giọng nói thông qua Voice Node. Lúc này dữ liệu âm thanh được truyền qua mạng Wi-Fi đến Local AI Server và được FastAPI tiếp nhận để thực hiện chuỗi xử lý gồm nhận dạng giọng nói, xử lý ngôn ngữ và tổng hợp câu trả lời. Dữ liệu từ Weather Node được sử dụng làm thông tin đầu vào bổ sung, giúp hệ thống có khả năng phản hồi các truy vấn liên quan đến điều kiện môi trường. Sau khi hoàn tất xử lý, kết quả âm thanh được gửi trở lại Voice Node để phát qua loa.

ABSTRACT

This project presents the research, design, and implementation of a virtual assistant system operating primarily within a local area network, aiming to reduce reliance on cloud-based AI services and to build a low-operating-cost solution suitable for personal hardware and IoT devices.

The system is built with four main components, including:

First, the Voice Node utilizes an ESP32-S3 microcontroller combined with an INMP441 digital microphone for audio capture and a MAX98357A I2S amplifier for audio playback.

Secondly, the Weather Node utilizes an ESP8266 combined with a BMP280 sensor to collect environmental data and provide temperature and pressure information to the system.

Third, a local AI server is deployed on a personal computer to act as the central processing unit it uses FastAPI to communicate with connected IoT nodes, employs Faster-Whisper for speech-to-text conversion, utilizes Qwen2.5 (deployed locally via Ollama) for language processing and response generation, and finally uses edge-tts and pydub for text-to-speech synthesis.

Fourth, Blynk is used to build an interface for remotely monitoring data and system status.

The system's operation begins when a user issues a voice command via the Voice Node. Audio data is transmitted over Wi-Fi to the Local AI Server, where FastAPI receives it to execute a processing pipeline comprising speech recognition, language processing, and response synthesis. Data from the Weather Node serves as supplementary input, enabling the system to respond to queries regarding environmental conditions. Once processing is complete, the resulting audio is sent back to the Voice Node for playback through the speaker.

MỤC LỤC

1.1.	Bối cảnh và lý do	12
1.1.1.	Hạn chế của trợ lý ảo phụ thuộc vào mô hình đám mây	12
1.1.2.	Cơ hội từ mô hình ngôn ngữ nhỏ	12
1.2.	Mục tiêu đề tài.....	12
1.3.	Đối tượng và phạm vi nghiên cứu	13
1.3.1.	Đối tượng nghiên cứu.....	13
1.3.2.	Phạm vi nghiên cứu	13
1.4.	Phương pháp nghiên cứu	13
1.4.1.	Phương pháp nghiên cứu lý thuyết.....	13
1.4.2.	Phương pháp nghiên cứu thực nghiệm	13
Chương 2.	CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN.....	15
2.1.	Tổng quan hệ thống	15
2.2.	Giao tiếp âm thanh số trên vi điều khiển.....	15
2.2.1.	Giao tiếp I2S.....	15
2.2.3.	Vi điều khiển ESP32-S3	15
2.2.4.	INMP441 và MAX98357A.....	16
2.3.	Chuyển giọng nói thành văn bản	16
2.3.1.	Mô hình Whisper	16
2.3.2.	Thư viện Faster-Whisper	16
2.4.	Mô hình ngôn ngữ lớn và suy luận cục bộ.....	16
2.4.1.	Mô hình Qwen2.5.....	16
2.4.2.	Nền tảng phục vụ suy luận cục bộ Ollama.....	17
2.5.	Tổng hợp giọng nói.....	17
2.5.1.	Mô hình tổng hợp giọng nói edge-tts	17
2.5.2.	Mô hình xử lý và chuyển đổi định dạng âm thanh Pydub.....	17

2.6. Cảm biến môi trường và giao tiếp IoT	17
2.6.1. Cảm biến áp suất BMP280	17
2.6.2. Giao thức HTTP trong IoT	17
2.6.3. Nền tảng giám sát từ xa Blynk	17
2.7. Khảo sát công trình/sản phẩm liên quan.....	18
2.7.1. Giải pháp thương mại chạy trên nền tảng đám mây	18
2.7.2. Giải pháp mã nguồn mở chạy cục bộ	18
2.7.3. Bảng so sánh và ưu điểm của đề tài	19
Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	21
3.1. Kiến trúc tổng thể hệ thống	21
3.1.1. Sơ đồ khối tổng quan	21
3.1.2. Sơ đồ Use Case và đặc tả	21
3.2. Thiết kế phần cứng	22
3.2.1. Sơ đồ nguyên lý đấu nối Voice Node	22
3.2.2. Sơ đồ nguyên lý đấu nối Weather Node	24
3.3. Thiết kế xử lý.....	24
3.3.1. Sơ đồ mạng và triển khai vật lý	24
3.3.2. Sơ đồ tuần tự luồng hội thoại chính.....	25
3.3.3. Sơ đồ trạng thái hệ thống.....	26
3.3.4. Sơ đồ thuật toán.....	27
3.4. Thiết kế kiến trúc phần mềm.....	27
3.4.1. Sơ đồ thành phần phía server	27
Chương 4. HIỆN THỰC VÀ KIỂM THỬ	29
4.1. Môi trường triển khai.....	29
4.1.1. Phần cứng sử dụng	29
4.1.2. Môi trường phần mềm, ngôn ngữ và thư viện.....	29

4.2.	Hiện thực Voice Node	32
4.2.1.	Cấu hình I2S thu âm	32
4.2.2.	Cấu hình I2S phát âm thanh và Ring Buffer	33
4.3.	Hiện thực Weather Node.....	34
4.3.1.	Đọc dữ liệu cảm biến BMP280	34
4.3.2.	Gửi dữ liệu định kỳ lên server.....	35
4.4.	Hiện thực Local AI Server	35
4.4.1.	Module chuyển giọng nói thành văn bản	35
4.4.2.	Module suy luận ngôn ngữ	36
4.4.3.	Module tổng hợp giọng nói và truyền phát theo đoạn	36
4.4.4.	Rate Limiter	37
4.4.5.	Emergency và Fallback Handler	37
4.5.	Tích hợp bảng điều khiển giám sát Blynk.....	38
4.6.	Kiểm thử hệ thống	39
4.6.1.	Kiểm thử module chuyển giọng nói thành văn bản	39
4.6.2.	Kiểm thử module suy luận ngôn ngữ lớn	39
4.6.3.	Kiểm thử tổng hợp giọng nói.....	40
4.6.4.	Kiểm thử module REST Client & Rate Limiter.....	40
4.6.5.	Kiểm thử tích hợp toàn hệ thống	41
4.6.6.	Bảng tra kiểm thử và kết quả.....	43
Chương 5.	KẾT QUẢ, ĐÁNH GIÁ VÀ KẾT LUẬN	44
5.1.	Kết quả thực hiện.....	44
5.2.	Sản phẩm	44
5.3.	Đánh giá hiệu năng hệ thống	46
5.3.1.	Độ trễ phản hồi từng giai đoạn	46
5.3.1.	Độ chính xác nhận diện giọng nói.....	46

5.3.2.	Độ chính xác phân tích ngữ cảnh.....	47
5.4.	Đánh giá quá trình thực hiện.....	47
5.4.1.	Phân tích nguyên nhân khi hệ thống sai	47
5.4.2.	So sánh với các giải pháp đã khảo	48
5.5.	Kết luận.....	49
5.5.1.	Hạn chế của đề tài	49
5.5.2.	Kế hoạch phát triển hệ thống trong tương lai nếu có thêm 3 tháng.....	50
TÀI LIỆU THAM KHẢO.....		52
PHỤ LỤC		54

DANH MỤC TỪ VIẾT TẮT

Từ 1: SLM: Small Language Model

Từ 2: LLM: Large Language Model

Từ 3: STT: Speech To Text

Từ 4: TTS: Text To Speech / gTTS : Google Text To Speech

Từ 5: API: Application Programming Interface

Từ 6: SRT: System Response Times

Từ 7: AI: Artificial Intelligence

Từ 8: I2S: Inter-IC Sound

Từ 9: HTTP: Hypertext Transfer Protocol

Từ 10: OOM: Out Of Memory

DANH MỤC HÌNH VẼ

Hình 3.1 Sơ đồ khối tổng quan hệ thống	21
Hình 3.2 Sơ đồ Use Case	22
Hình 3.3 Bản thiết kế phần cứng Voice Node	23
Hình 3.4 Bản thiết kế phần cứng Weather Node.....	24
Hình 3.5 Sơ đồ mạng và triển khai vật lý	25
Hình 3.6 Sơ đồ tuần tự hệ thống	25
Hình 3.7 Sơ đồ trạng thái.....	26
Hình 3.8 Sơ đồ thuật toán hệ thống	27
Hình 3.9 Sơ đồ kiến trúc phần mềm	28
Hình 5.1 Hình ảnh sản phẩm thực tế của Voice Node (esp32s3n16r8)	45
Hình 5.2 Hình ảnh sản phẩm thực tế của Weather Node (esp8266).....	45
Hình 5.3 Hình ảnh sản phẩm thực tế của toàn bộ hệ thống	46

DANH MỤC BẢNG

Bảng 2.1	Bảng so sánh các công trình liên quan.....	19
Bảng 3.1	Bảng cấu hình chân GPIO Voice Node.....	23
Bảng 3.2	Bảng cấu hình chân GPIO Weather Node	24
Bảng 4.1	Bảng liệt kê các phần mềm, ngôn ngữ và thư viện sử dụng	29
Bảng 4.2	Bảng so sánh các thành phần khác so với thành phần đã chọn	29
Bảng 4.3	Bảng định nghĩa Virtual Pin và nguyên lý hoạt động	39
Bảng 4.4	Bảng đánh giá tổng quan hệ thống sau kiểm thử	43
Bảng 5.1	Bảng so sánh đồ án sau thực hiện so với mục tiêu đề ra ban đầu	44
Bảng 5.2	Bảng nhận xét độ trễ của chu trình tích hợp	46
Bảng 5.3	Bảng so sánh giữa giải pháp của đề tài và các giải pháp khác	48

Chương 1. GIỚI THIỆU

1.1. Bối cảnh và lý do

Các trợ lý mô hình ngôn ngữ lớn thương mại như: ChatGPT, Gemini, Claude, .v.v đã nhanh chóng trở thành một phần không thể thiếu trong cuộc sống hiện đại, từ hỗ trợ soạn thảo văn bản đến phản hồi ý tưởng. Để mang lại trải nghiệm, tối ưu nhất cho người dùng các mô hình này đều được xử lý trên các hạ tầng đám mây của nhà cung cấp dịch vụ dẫn đến việc phát sinh ra nhiều chi phí để vận hành và duy trì, độ trễ mạng khiến và trên hết là rủi ro quyền riêng tư dữ liệu giọng nói người dùng. Từ những băn khoăn đó sự phát triển của các mô hình ngôn ngữ nhỏ được ra đời với mục đích phù hợp chạy độc lập trên máy chủ cá nhân cùng các kỹ thuật tối ưu suy luận đã giải quyết được vấn đề chi phí, độ trễ mạng và trên hết là bảo mật được về dữ liệu người dùng.

1.1.1. Hạn chế của trợ lý ảo phụ thuộc vào mô hình đám mây

Mô hình đám mây tồn tại ba hạn chế đáng chú ý sau:

- _ Dữ liệu giọng nói đã và đang là loại dữ liệu sinh trắc học nhạy cảm phải được truyền tải và lưu trữ trên hạ tầng của bên thứ ba, làm phát sinh rủi ro về quyền riêng tư.
- _ Độ trễ phản hồi phụ thuộc vào chất lượng và độ ổn định của kết nối mạng giữa thiết bị và dịch vụ đám mây. Khi đường truyền không ổn định hoặc băng thông thấp khiến thời gian truyền dữ liệu và nhận kết quả có thể tăng từ đó ảnh hưởng đến trải nghiệm tương tác thời gian thực.
- _ Việc vận hành liên tục dịch vụ đám mây phát sinh chi phí cho cả nhà cung cấp lẫn người dùng khi mở rộng quy mô.

1.1.2. Cơ hội từ mô hình ngôn ngữ nhỏ

Trong những năm gần đây, sự phát triển của các mô hình ngôn ngữ lớn nhưng có kích thước nhỏ điển hình là các mô hình trong họ Qwen2.5 [1] với số lượng tham số từ 0,5 đến 7 tỷ đã cho thấy khả năng đạt chất lượng suy luận chấp nhận được trong khi có thể triển khai trên phần cứng phổ thông.

1.2. Mục tiêu đề tài

_ Thiết kế và thi công phần cứng thu-phát âm thanh thời gian thực trên vi điều khiển ESP32-S3, đảm bảo giao tiếp ổn định qua chuẩn I2S với micro INMP441 và bộ khuếch đại MAX98357A.

_ Xây dựng quy trình xử lý trí tuệ nhân tạo cục bộ hoàn chỉnh gồm: nhận dạng giọng nói, suy luận ngôn ngữ và tổng hợp giọng nói, không phụ thuộc vào dịch vụ trí tuệ nhân tạo trên nền tảng đám mây cho phần suy luận cốt lõi.

_ Tích hợp dữ liệu cảm biến môi trường thời gian thực từ vi mạch ESP8266 sử dụng HW-611 làm cơ sở ngữ cảnh cho mô hình ngôn ngữ, giúp hệ thống trả lời được các câu hỏi liên quan đến điều kiện môi trường thực tế.

_ Xây dựng cơ chế giám sát và điều khiển từ xa hệ thống qua ứng dụng Blynk.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm: kiến trúc hệ thống trợ lý ảo, các kỹ thuật tối ưu hóa suy luận mô hình ngôn ngữ lớn trên phần cứng hạn chế tài nguyên, và giao tiếp âm thanh số thời gian thực qua chuẩn I2S trên vi điều khiển.

1.3.2. Phạm vi nghiên cứu

Đề tài giới hạn phạm vi triển khai trong các điều kiện sau: hệ thống phục vụ cho một người dùng tại một thời điểm, hoạt động trong phạm vi một mạng cục bộ duy nhất, sử dụng mô hình ngôn ngữ có kích thước từ 1,5 đến 7 tỷ tham số phù hợp với cấu hình phần cứng phổ thông đang có.

Đề tài không thực hiện: xử lý đa người dùng đồng thời, tinh chỉnh mô hình ngôn ngữ, và huấn luyện lại mô hình nhận dạng giọng nói.

1.4. Phương pháp nghiên cứu

1.4.1. Phương pháp nghiên cứu lý thuyết

Được thực hiện thông qua khảo sát tài liệu kỹ thuật chính thức, tài liệu hướng dẫn của nhà sản xuất linh kiện, và các công trình khoa học liên quan đến kiến trúc mô hình ngôn ngữ, xử lý tiếng nói và giao tiếp nhúng thời gian thực.

1.4.2. Phương pháp nghiên cứu thực nghiệm

Phương pháp nghiên cứu thực nghiệm được thực hiện thông qua thiết kế, lắp ráp và kiểm thử phần cứng đồng thời triển khai các thành phần phần mềm cho ESP32-S3 và máy tính cục bộ. Hệ thống được kiểm thử theo từng chức năng bao gồm: thu âm, truyền dữ liệu, nhận dạng giọng nói, xử lý câu hỏi bằng mô hình ngôn ngữ, tổng hợp giọng nói và phát âm thanh từ đó làm cơ sở đánh giá, phát hiện lỗi và hiệu chỉnh thiết kế.

Chương 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN

2.1. Tổng quan hệ thống

Trợ lý ảo Edge AI là thuật ngữ chỉ các hệ thống trợ lý ảo mà phần lớn hoặc toàn bộ quá trình xử lý trí tuệ nhân tạo được thực hiện trên thiết bị của người dùng hoặc trên một máy chủ trong cùng mạng cục bộ thay vì trên hạ tầng đám mây. Kiến trúc này đòi hỏi giải quyết đồng thời ba bài toán kỹ thuật:

_ Thu và phát âm thanh chất lượng cao trên phần cứng nhúng tài nguyên hạn chế hiện có.

_ Triển khai chuỗi xử lý trí tuệ nhân tạo như: nhận dạng giọng nói, suy luận ngôn ngữ, tổng hợp giọng nói trên phần cứng phổ thông với độ trễ chấp nhận được.

_ Tích hợp dữ liệu ngữ cảnh thời gian thực để mô hình ngôn ngữ có thể trả lời chính xác các câu hỏi liên quan đến trạng thái thực tế của môi trường xung quanh.

2.2. Giao tiếp âm thanh số trên vi điều khiển

2.2.1. Giao tiếp I2S

Là giao tiếp nối tiếp đồng bộ được thiết kế chuyên biệt cho truyền tải dữ liệu âm thanh số giữa các vi mạch, sử dụng ba tín hiệu chính: tín hiệu xung nhịp bit từ: BCLK, SCK đồng bộ hóa việc dịch từng bit dữ liệu, tín hiệu lựa chọn từ: WS, LRC xác định kênh trái hoặc phải đang được truyền, và tín hiệu dữ liệu từ: SD, DIN, DOUT mang giá trị mẫu âm thanh dạng số.

2.2.2. Giao thức giao tiếp I2C

Giao thức sử dụng hai đường tín hiệu dùng chung cho toàn bộ thiết bị trên bus: đường dữ liệu nối tiếp và đường xung nhịp nối tiếp, cho phép nhiều thiết bị ngoại vi giao tiếp với một hoặc nhiều vi điều khiển trên cùng một cặp dây vật lý, mỗi thiết bị được định danh bằng một địa chỉ riêng thay vì cần một đường tín hiệu chọn chip riêng như chuẩn SPI [2].

2.2.3. Vi điều khiển ESP32-S3

ESP32-S3 là vi điều khiển hai lõi kiến trúc Xtensa LX7 do Espressif Systems phát triển, tích hợp sẵn kết nối Wi-Fi và Bluetooth Low Energy, hỗ trợ hai bộ điều khiển ngoại vi I2S độc lập cho phép đồng thời thu và phát âm thanh trên hai kênh riêng biệt.

2.2.4. INMP441 và MAX98357A

Micro kỹ thuật số INMP441 tích hợp sẵn bộ chuyển đổi tương tự sang số, xuất dữ liệu trực tiếp qua giao tiếp I2S với độ phân giải 24-bit nhưng được đặt trong khung I2S là 32-bit [4].

MAX98357A là bộ khuếch đại công suất lớp D tích hợp bộ giải mã I2S, nhận trực tiếp luồng dữ liệu âm thanh số và khuếch đại ra loa mà không cần bộ chuyển đổi số sang tương tự rời [5].

2.3. Chuyển giọng nói thành văn bản

2.3.1. Mô hình Whisper

Là mô hình nhận dạng giọng nói tự động được OpenAI công bố năm 2022, huấn luyện trên 680.000 giờ dữ liệu âm thanh đa ngôn ngữ thu thập từ Internet theo phương pháp giám sát yếu [6]. Mô hình sử dụng kiến trúc dạng mã hóa và giải mã, cho phép thực hiện đồng thời nhận dạng giọng nói, phát hiện ngôn ngữ và dịch thuật.

2.3.2. Thư viện Faster-Whisper

Là một thư viện triển khai Whisper dựa trên CTranslate2, được sử dụng để chuyển đổi tiếng nói thành văn bản đồng thời được tối ưu để suy luận hiệu quả trên CPU và GPU, hỗ trợ các kỹ thuật như lượng tử hóa để giảm thời gian suy luận và mức sử dụng bộ nhớ [7].

2.4. Mô hình ngôn ngữ lớn và suy luận cục bộ

2.4.1. Mô hình Qwen2.5

Là họ mô hình ngôn ngữ lớn do Alibaba phát triển, cung cấp nhiều phiên bản với số lượng tham số khác nhau, được huấn luyện trên tập dữ liệu quy mô lớn gồm 18 nghìn tỷ token [1]. Các phiên bản kích thước nhỏ trong họ mô hình này từ 1,5 đến 7 tỷ tham số phù hợp để triển khai suy luận trên phần cứng cá nhân.

2.4.2. Nền tảng phục vụ suy luận cục bộ Ollama

Là nền tảng mã nguồn mở cho phép tải, quản lý và phục vụ suy luận các mô hình ngôn ngữ lớn trên phần cứng CPU hoặc GPU phổ thông thông qua giao diện lập trình ứng dụng cục bộ, được xây dựng trên nền llama.cpp [10].

2.5. Tổng hợp giọng nói

2.5.1. Mô hình tổng hợp giọng nói edge-tts

Là thư viện Python mã nguồn mở cho phép gọi dịch vụ tổng hợp giọng nói trực tuyến của Microsoft Edge trực tiếp từ mã Python, mà không cần cài đặt trình duyệt Edge, hệ điều hành Windows, hay khóa API riêng. Thư viện cung cấp giao diện lập trình bất đồng bộ cho phép tùy chỉnh giọng đọc, tốc độ nói, âm lượng và cao độ thông qua các tham số truyền vào, hỗ trợ nhiều ngôn ngữ bao gồm tiếng Việt [8].

2.5.2. Mô hình xử lý và chuyển đổi định dạng âm thanh Pydub

Là thư viện Python xử lý âm thanh mã nguồn mở, dựa trên FFmpeg ở tầng dưới để giải mã MP3 và chuyển đổi về định dạng PCM tuyến tính 16-bit, 16kHz, đơn kênh, đúng định dạng ESP32-S3 phát được trực tiếp không cần giải mã thêm [3].

2.6. Cảm biến môi trường và giao tiếp IoT

2.6.1. Cảm biến áp suất BMP280

Là cảm biến áp suất khí quyển và nhiệt độ dạng số do Bosch Sensortec phát triển, giao tiếp qua chuẩn I2C hoặc SPI [9].

2.6.2. Giao thức HTTP trong IoT

HTTP là giao thức tầng ứng dụng được chuẩn hóa bởi IETF. HTTP là giao thức stateless, sử dụng mô hình request hoặc response giữa client và server. Client gửi một HTTP request tới server, server xử lý và trả về HTTP response [11].

2.6.3. Nền tảng giám sát từ xa Blynk

Blynk là nền tảng phát triển ứng dụng IoT cung cấp giao diện điều khiển và giám sát trên thiết bị di động, giao tiếp với thiết bị nhúng thông qua các “virtual pin” và giao diện lập trình ứng dụng REST [12].

2.7. Khảo sát công trình/sản phẩm liên quan

2.7.1. Giải pháp thương mại chạy trên nền tảng đám mây

Về nguyên lý, người dùng phát ra câu lệnh bằng giọng nói thông qua điện thoại, loa thông minh hoặc thiết bị có tích hợp Google Assistant. Hệ thống tiếp nhận tín hiệu âm thanh, thực hiện các bước xử lý liên quan đến nhận dạng và hiểu ngôn ngữ, sau đó xác định ý định của người dùng và thực hiện hành động tương ứng.

Với các ứng dụng được mở rộng bởi bên thứ ba, Google Assistant có thể chuyển thông tin đã được nhận dạng tới dịch vụ xử lý của ứng dụng thông qua cơ chế như: fulfillment hoặc webhook để đưa ra kết quả sau đó được trả về để tạo phản hồi bằng giọng nói hoặc giao diện trực quan.

Luồng xử lý được khái quát như sau:

Người dùng => Microphone => Voice processing hoặc Google Assistant => NLU hoặc Intent => Fulfillment hoặc dịch vụ Google => Response => TTS => Loa.

Tuy nhiên, mô hình phụ thuộc vào hạ tầng của nhà cung cấp cũng tạo ra vấn đề về quyền riêng tư và khả năng kiểm soát dữ liệu. Tài liệu của Google cho biết dữ liệu liên quan đến tương tác với Assistant có thể được xử lý và chia sẻ trong những trường hợp phục vụ hoạt động của dịch vụ hoặc ứng dụng bên thứ ba. Ngoài ra, người dùng có thể bật tính năng lưu hoạt động giọng nói và âm thanh trong tài khoản Google. Các nghiên cứu về trợ lý giọng nói cũng chỉ ra rằng việc truyền dữ liệu thoại tới hệ thống đám mây làm phát sinh các vấn đề về quyền riêng tư và kiểm soát dữ liệu người dùng [13], [14].

2.7.2. Giải pháp mã nguồn mở chạy cục bộ

Để giảm sự phụ thuộc vào dịch vụ đám mây, một hướng tiếp cận khác là triển khai trợ lý giọng nói trên hạ tầng cục bộ. Đại diện tiêu biểu là Home Assistant Assist kết hợp với các linh kiện như: ESP32-S3 hoặc các thiết bị tương tự.

Luồng xử lý được khái quát như sau:

Microphone => Wake Word => STT => Intent hoặc NLU => Home Assistant => TTS => Speaker

Điểm khác biệt quan trọng là các thành phần này có thể được triển khai hoàn toàn trong mạng nội bộ đưa khả năng bảo mật thông tin người dùng lên hàng đầu.

2.7.3. Bảng so sánh và ưu điểm của đề tài

Bảng 2.1 Bảng so sánh các công trình liên quan

Tiêu chí	Google Assistant	Home Assistant	LEVA
Kiến trúc	Chủ yếu dựa trên hệ sinh thái dịch vụ của Google	Pipeline có thể chạy cục bộ hoặc kết hợp cloud	Edge device + local AI server
Độ trễ phản hồi	Phụ thuộc Internet và hạ tầng cloud	Có thể thấp hơn khi xử lý trong LAN, nhưng phụ thuộc năng lực máy chủ	Hướng tới giảm độ trễ nhờ xử lý cục bộ và LAN
Chi phí vận hành	Phụ thuộc nền tảng hoặc dịch vụ của nhà cung cấp	Không bắt buộc thuê cloud, cần phần cứng cục bộ	Chủ yếu là chi phí đầu tư phần cứng và máy chủ cục bộ
Bảo vệ quyền riêng tư	Thấp hơn mô hình fully-local do dữ liệu có thể được xử lý bởi dịch vụ bên ngoài	Cao khi sử dụng fully-local	Cao, hướng tới giữ dữ liệu xử lý trong mạng nội bộ
Khả năng tùy biến	Bị ràng buộc bởi hệ sinh thái của Google	Cao, có thể thay thế STT hoặc TTS hoặc wake-word hoặc LLM	Cao, chủ động kiểm soát từng thành phần
Hoạt động khi mất Internet	Hạn chế	Có thể hoạt động với pipeline hoàn toàn cục bộ	Là mục tiêu thiết kế
LLM tùy chỉnh	Không kiểm soát trực tiếp mô hình lỗi	Có thể tích hợp LLM cục bộ	Có thể sử dụng Qwen hoặc LLM cục bộ tùy cấu hình
Kiểm soát dữ liệu	Phụ thuộc chính sách nhà cung cấp	Người dùng kiểm soát hạ tầng cục bộ	Người dùng kiểm soát toàn bộ pipeline cục bộ
Ưu điểm	Khả năng xử lý ngôn ngữ tự nhiên tốt, dịch vụ đa dạng, dễ tích hợp với các thiết bị và dịch vụ của Google	Mã nguồn mở, khả năng tùy biến cao, có thể triển khai local	Chi phí thiết bị đầu cuối thấp, chủ động kiểm soát quy trình, có thể sử dụng LLM tùy chỉnh, hoàn toàn nội bộ

Nhược điểm	Phụ thuộc vào Internet và hạ tầng cloud, khả năng kiểm soát dữ liệu và mô hình lỗi bị giới hạn, khó tùy chỉnh sâu	Việc triển khai local yêu cầu người dùng tự chuẩn bị và quản lý phần cứng, hiệu năng phụ thuộc vào máy chủ cục bộ	Hiệu năng phụ thuộc vào máy chủ AI cục bộ, cần tự triển khai và bảo trì nhiều thành phần vật lý cả hệ thống số ảo, phạm vi hoạt động phụ thuộc mạng LAN
Độ trễ	Phụ thuộc vào chất lượng mạng và thời gian xử lý trên cloud	Có thể giảm khi xử lý local nhưng phụ thuộc năng lực phần cứng	Hướng tới giảm độ trễ truyền tải nhờ LAN và xử lý AI cục bộ

Từ khảo sát trên có thể nhận thấy hai nhóm giải pháp hiện nay tồn tại sự đánh đổi tương đối rõ ràng.

Nhóm thương mại dựa trên cloud như Google Assistant có ưu thế về khả năng xử lý AI mạnh, khả năng mở rộng và hệ sinh thái dịch vụ phong phú, nhưng phụ thuộc vào Internet và hạ tầng của nhà cung cấp làm giảm khả năng kiểm soát dữ liệu và khả năng tùy chỉnh sâu của người dùng.

Nhóm mã nguồn mở và xử lý cục bộ như Home Assistant lại giải quyết tốt hơn vấn đề riêng tư và khả năng tùy biến nhưng hiệu năng vẫn phụ thuộc đáng kể vào tài nguyên phần cứng cục bộ. Khi sử dụng các mô hình STT, TTS hoặc LLM lớn thời gian xử lý có thể trở thành yếu tố hạn chế.

LEVA hướng tới khoảng trống nằm giữa hai nhóm trên: xây dựng một trợ lý giọng nói có thiết bị đầu cuối chi phí thấp dựa trên ESP32-S3, kết hợp với máy chủ AI cục bộ để thực hiện các tác vụ xử lý nặng như STT và LLM, đồng thời sử dụng kết nối mạng nội bộ thay vì phụ thuộc hoàn toàn vào cloud. Kiến trúc này nhằm đạt được sự cân bằng giữa độ trễ, quyền riêng tư, chi phí và khả năng tùy biến.

Luồng xử lý của LEVA được khái quát như sau:

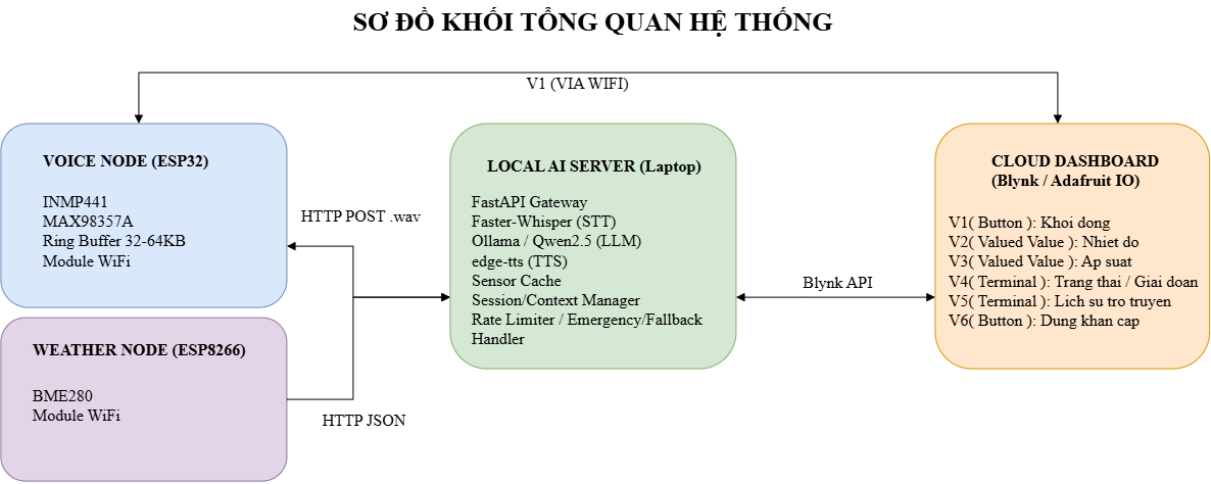
Người dùng => Microphone => ESP32-S3 => xử lý âm thanh => LAN => STT cục bộ => LLM cục bộ => TTS cục bộ => ESP32-S3 => Speaker

Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống

3.1.1. Sơ đồ khối tổng quan

Hệ thống được thiết kế theo kiến trúc phân tán gồm bốn khối chức năng độc lập, giao tiếp với nhau qua mạng cục bộ: Voice Node đảm nhiệm thu-phát âm thanh, Weather Node đảm nhiệm thu thập dữ liệu môi trường, Local AI Server đảm nhiệm toàn bộ xử lý trí tuệ nhân tạo và Cloud Dashboard đảm nhiệm giám sát và điều khiển từ xa như trong Hình 3.1.

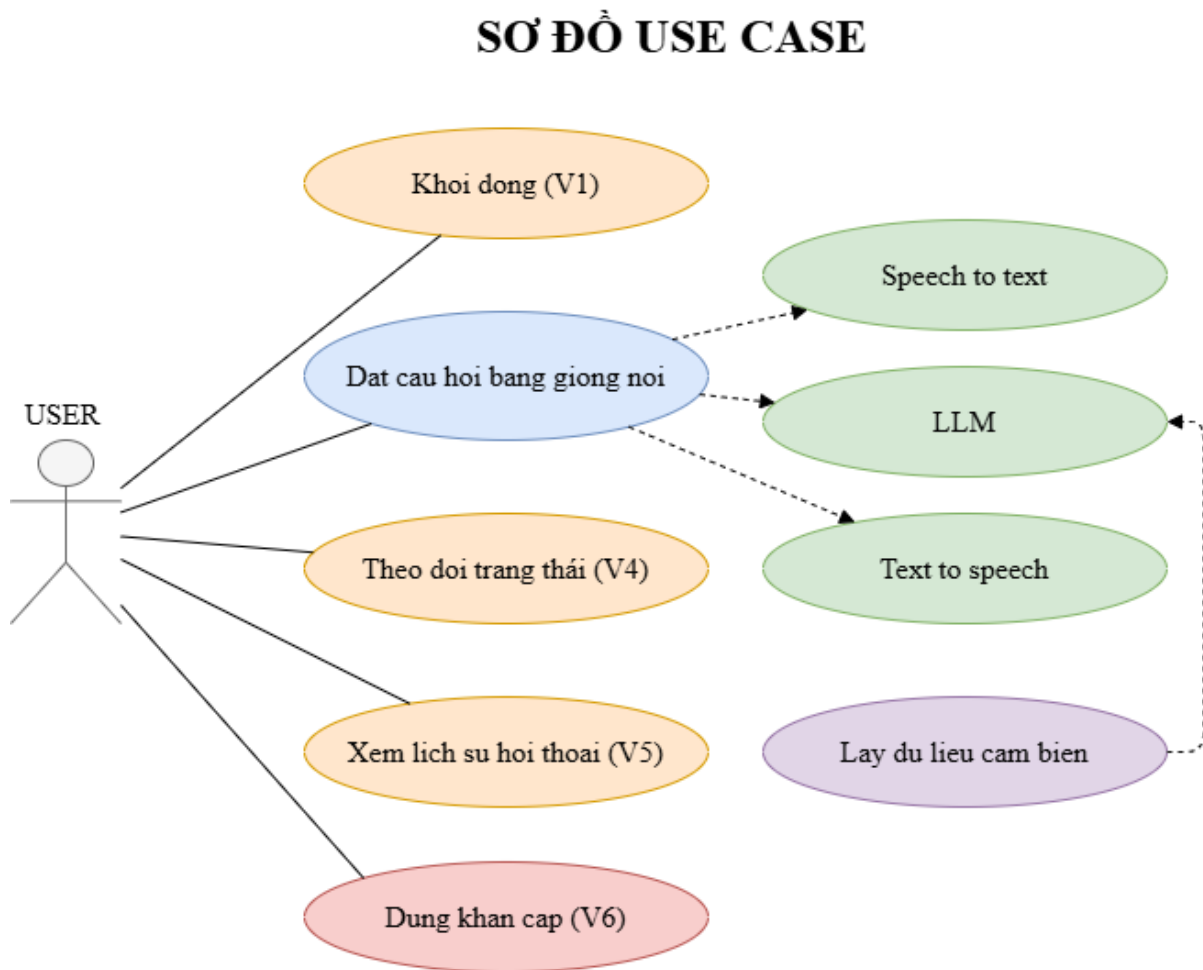


Hình 3.1 Sơ đồ khối tổng quan hệ thống

Trong hình trên việc phân chia thành các khối độc lập cho phép mỗi thành phần được phát triển, kiểm thử và thay thế riêng biệt mà không ảnh hưởng đến toàn hệ thống. Dữ liệu âm thanh trao đổi hai chiều giữa Voice Node và Local AI Server qua hai chiều: chiều lên qua giao thức http post mang tệp âm thanh định dạng wav, chiều xuống qua http truyền phát theo đoạn mang dữ liệu âm thanh số thô. Weather Node gửi định kỳ dữ liệu cảm biến dạng json lên Local AI Server để lưu vào bộ nhớ đệm, phục vụ truy xuất tức thời khi có câu hỏi liên quan. Local AI Server đồng bộ hai chiều với Cloud Dashboard gửi cập nhật trạng thái, số token và lịch sử hội thoại, nhận lệnh kích hoạt và lệnh dừng khẩn cấp.

3.1.2. Sơ đồ Use Case và đặc tả

Người dùng sử dụng nút bấm ảo trên nền tảng Blynk tương tác trực tiếp để kích hoạt trợ lý, đặt câu hỏi, xem lịch sử hội thoại và theo dõi trạng thái và gia đoạn được thể hiện qua Hình 3.2.

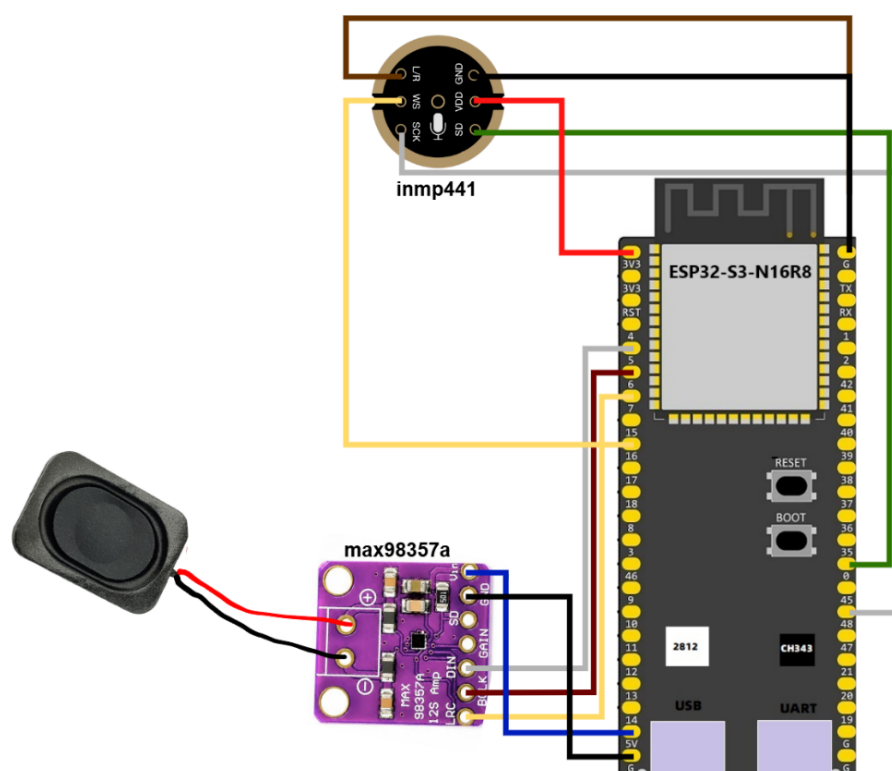


Hình 3.2 Sơ đồ Use Case

3.2. Thiết kế phần cứng

3.2.1. Sơ đồ nguyên lý đầu nối Voice Node

Voice Node sử dụng vi điều khiển ESP32-S3-N16R8. Cấu hình chân được xác định trong Bảng 3.1 và Bảng 3.2. Ngoài ra, mô hình thực tế được mô phỏng ở Hình 3.3 và Hình 3.4.

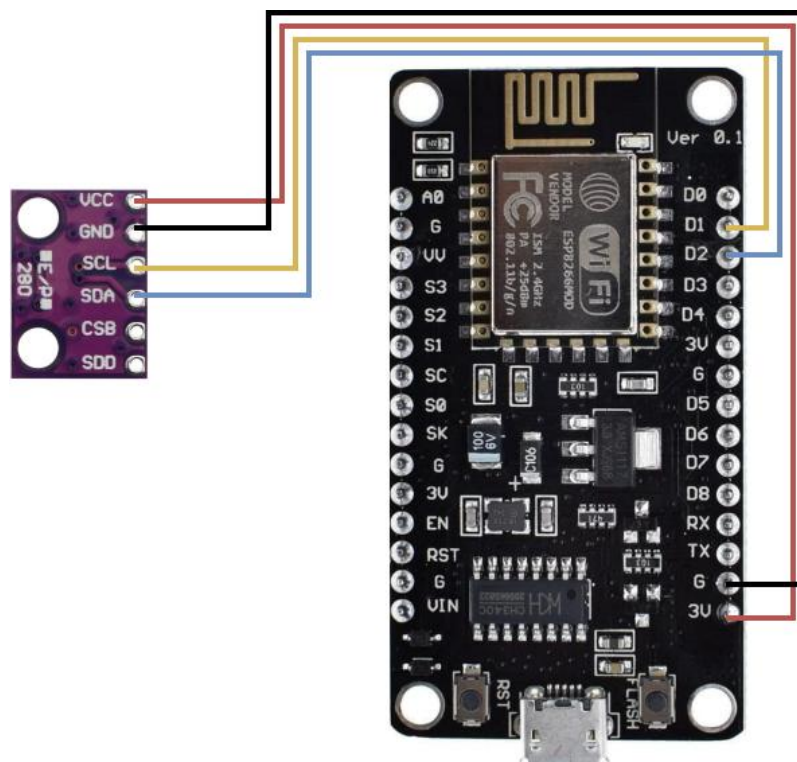


Hình 3.3 Bản thiết kế phần cứng Voice Node

Bảng 3.1 Bảng cấu hình chân GPIO Voice Node

Tín hiệu	Chân GPIO	Ghi chú
Mic INMP441 - WS	GPIO15	Word Select
Mic INMP441 - SD	GPIO35	Dữ liệu vào
Mic INMP441 - SCK	GPIO45	Xung nhịp bit
Mic INMP441 - VDD	3V3	Cấp nguồn 3.3V
Mic INMP441 - GND	G	Mass chung
Loa MAX98357A - BCLK	GPIO5	Bit Clock
Loa MAX98357A - LRC	GPIO6	Chọn kênh
Loa MAX98357A - DIN	GPIO4	Dữ liệu ra
Loa MAX98357A - VIN	5V	Cấp nguồn 5V
Loa MAX98357A - GND	G	Mass chung

3.2.2. Sơ đồ nguyên lý đấu nối Weather Node



Hình 3.4 Bản thiết kế phần cứng Weather Node

Bảng 3.2 Bảng cấu hình chân GPIO Weather Node

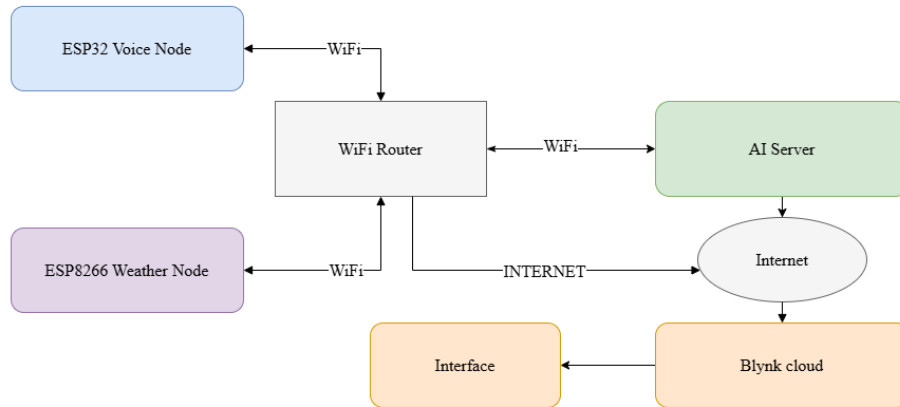
Tín hiệu	Chân GPIO	Ghi chú
BMP280 - SCL	D1	Bit Clock
BMP280 - SDA	D2	Truyền nhận dữ liệu
BMP280 - VCC	3V	Cấp nguồn 3V
BMP280 - GND	G	Mass chung

3.3. Thiết kế xử lý

3.3.1. Sơ đồ mạng và triển khai vật lý

Voice Node, Weather Node và Local AI Server cùng kết nối vào một router WiFi trong mạng LAN nội bộ. Local AI Server kết nối ra Internet để đồng bộ với Blynk Cloud, thiết bị di động chạy ứng dụng Blynk có thể kết nối từ bất kỳ mạng nào miễn có Internet hệ thống trên được trình bày qua Hình 3.5.

SƠ ĐỒ MẠNG VÀ TRIỂN KHAI

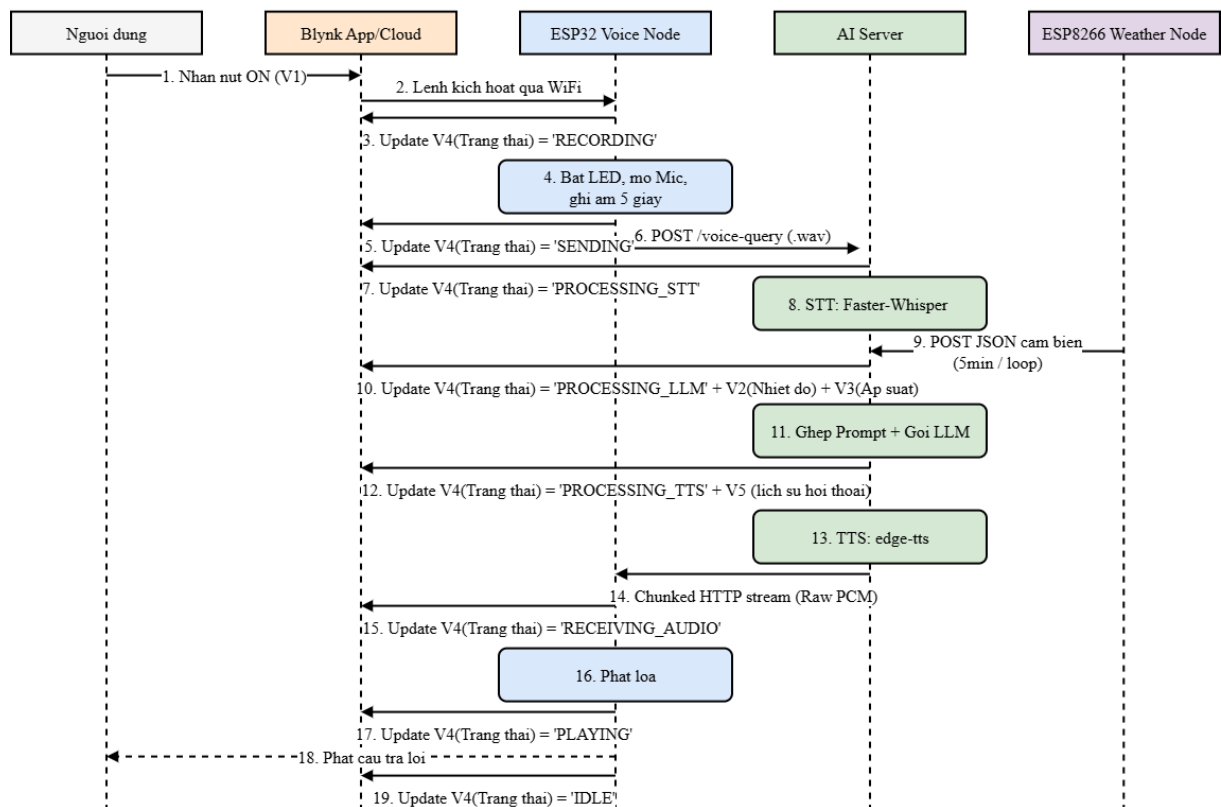


Hình 3.5 Sơ đồ mạng và triển khai vật lý

3.3.2. Sơ đồ tuần tự luồng hội thoại chính

Sơ đồ tuần tự mô tả trình tự trao đổi thông điệp giữa năm thực thể: Người dùng, Blynk App hoặc Cloud, ESP32 Voice Node, Local AI Server và ESP8266 Weather Node, theo thứ tự thời gian từ lúc kích hoạt đến lúc phát âm thanh trả lời. Đáng chú ý, việc cập nhật dữ liệu cảm biến từ Weather Node diễn ra độc lập theo chu kỳ 5 phút, không phụ thuộc vào luồng xử lý chính, được biểu diễn bằng khung lặp (loop) riêng trong Hình 3.6.

SƠ ĐỒ TUẦN TỰ HỆ THỐNG

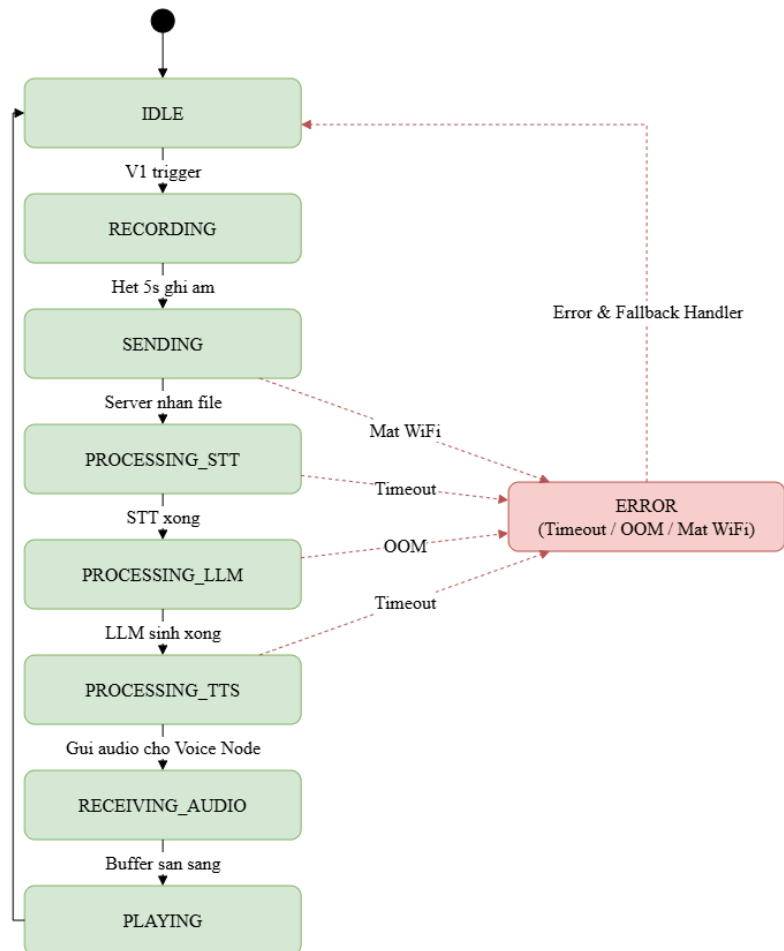


Hình 3.6 Sơ đồ tuần tự hệ thống

3.3.3. Sơ đồ trạng thái hệ thống

Hệ thống được mô hình hóa qua tám trạng thái chính sau: IDLE chờ lệnh, RECORDING ghi âm, SENDING gửi file ghi âm lên sever, PROCESSING_STT xử lý chuyển hóa từ giọng nói sang văn bản, PROCESSING_LLM AI suy nghĩ, PROCESSING_TTS xử lý chuyển hóa từ văn bản sang giọng nói, STREAMING_AUDIO phát luồng âm thanh và PLAYING phát âm thanh, cùng một trạng thái ERROR dùng chung cho các tình huống bất thường như: mất kết nối mạng, hết thời gian xử lý, hết bộ nhớ. Từ mọi trạng thái xử lý, hệ thống đều có đường chuyển tiếp sang ERROR, và từ ERROR luôn có đường quay về IDLE thông qua cơ chế ngắt mạch hoặc lệnh dừng khẩn cấp từ người dùng, đảm bảo hệ thống không bị treo vĩnh viễn ở một trạng thái lỗi, như trình bày trong Hình 3.7.

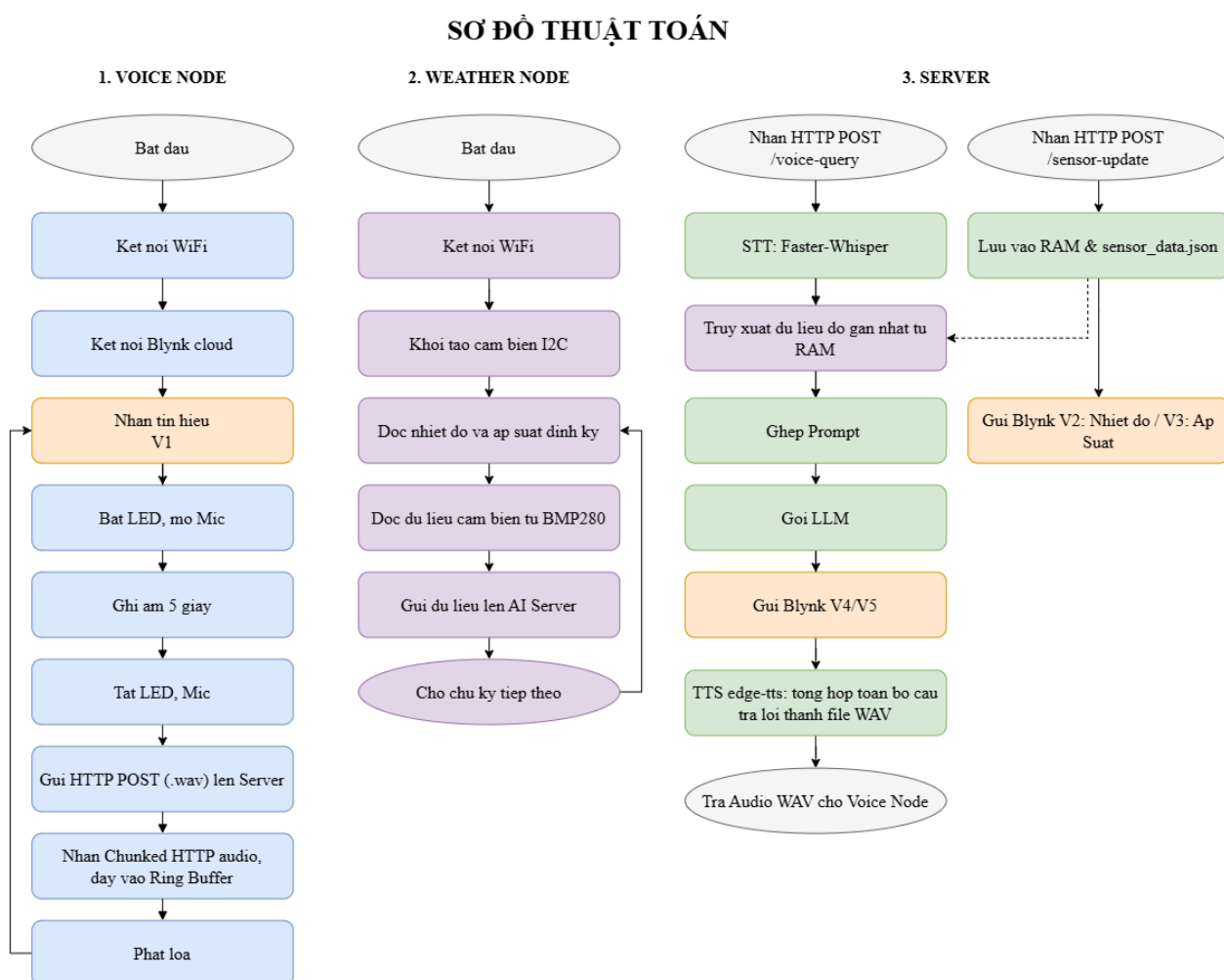
SƠ ĐỒ TRẠNG THÁI



Hình 3.7 Sơ đồ trạng thái

3.3.4. Sơ đồ thuật toán

Lưu đồ thuật toán được xây dựng riêng cho ba thành phần: Voice Node vòng lặp chờ tín hiệu kích hoạt, ghi âm, gửi dữ liệu, nhận và phát luồng âm thanh trả về và tiến trình xử lý phía SERVER nhận dạng giọng nói, ghép ngữ cảnh cảm biến, gọi mô hình ngôn ngữ LLM tự quyết định mức độ liên quan của ngữ cảnh, tổng hợp giọng nói theo từng câu. Ngoài ra, Weather Node đóng vai trò gửi dữ liệu vòng lặp liên tục nếu hệ thống còn chạy cho phía sever như trình bày trong Hình 3.8.



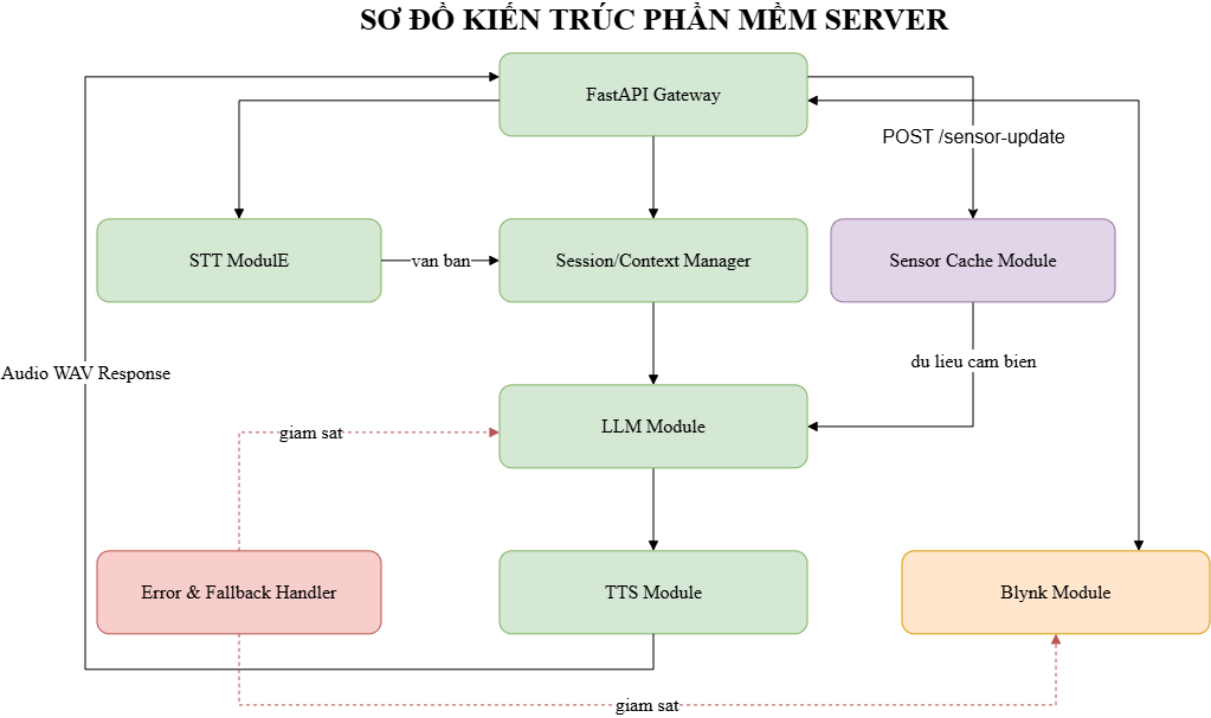
Hình 3.8 Sơ đồ thuật toán hệ thống

3.4. Thiết kế kiến trúc phần mềm

3.4.1. Sơ đồ thành phần phía server

Phần mềm phía Local AI Server được tổ chức thành tám thành phần độc lập: FastAPI Gateway là điểm vào giao tiếp HTTP, STT Module sử dụng Faster-Whisper, Session hoặc Context Manager quản lý lịch sử hội thoại và cấu trúc prompt, Sensor Cache

Module để lưu dữ liệu cảm biến mới nhất, LLM Module sử dụng Ollama phục vụ mô hình Qwen2.5, TTS Module sử dụng thư viện edge-tts, Blynk Module đồng bộ bảng điều khiển và Emergency Handler có nhiệm vụ giám sát và xử lý lỗi như đã được trình bày trong Hình 3.9.



Hình 3.9 Sơ đồ kiến trúc phần mềm

Chương 4. HIỆN THỰC VÀ KIỂM THỬ

4.1. Môi trường triển khai

4.1.1. Phần cứng sử dụng

Phần cứng thực tế sử dụng gồm:

VOICE NODE: 01 board ESP32-S3-N16R8 (16MB Flash hoặc 8MB PSRAM Octal), 01 module micro INMP441, 01 module khuếch đại MAX98357A kèm loa mini 3W 8ohm, breadboard, dây jumper.

WEATHER NODE: 01 board ESP8266 NodeMCU, 01 module cảm biến HW-611 (BMP280), breadboard, dây jumper và nguồn cấp 3V.

4.1.2. Môi trường phần mềm, ngôn ngữ và thư viện

Bảng 4.1 Bảng liệt kê các phần mềm, ngôn ngữ và thư viện sử dụng

Thành phần	Phiên bản / Công cụ
IDE lập trình vi điều khiển	Arduino IDE 2.3.10, gói board ESP32 (Espressif Systems) v3.3.11, Adafruit_BMP280, Adafruit_Sensor, BlynkSimpleEsp32
Ngôn ngữ firmware	C hoặc C++ (ESP-IDF I2S driver, thư viện ESP8266Audio)
Ngôn ngữ server	Python 3, framework FastAPI, uvicorn
Module STT	Faster-Whisper
Module LLM	Ollama, mô hình Qwen2.5
Module TTS	Edge-tts, pydub
Bảng điều khiển	Blynk IoT

Bảng 4.2 Bảng so sánh các thành phần khác so với thành phần đã chọn

Thành phần đã chọn	So sánh với	Lý do sử dụng	Ưu điểm so với thành phần khác	Nhược điểm / Đánh đổi	Nguồn
Faster-Whisper	OpenAI Whisper gốc, whisper.cpp	Cần suy luận STT trên CPU cá nhân với độ trễ chấp nhận	Trên cùng cấu hình CPU Intel Xeon Gold 6226R, 8 luồng,	Trên CPU thuần, whisper.cpp vẫn tiết kiệm	[7], [15]

		được, hỗ trợ lượng tử hóa int8 để giảm RAM	mô hình small chạy int8 chỉ mất 2 phút 04 giây, nhanh hơn nhiều so với Whisper gốc fp32 10 phút 31 giây và whisper.cpp fp16 12 phút 39 giây khi giải mã cùng 13 phút âm thanh, giảm ~50% RAM nhờ int8	RAM hơn faster-whisper trong một số benchmark độc lập gần đây, phải cài thêm CTranslate2 làm phụ thuộc ngoài Trên CPU thuần, whisper.cpp vẫn tiết kiệm RAM hơn faster-whisper trong một số benchmark độc lập gần đây, phải cài thêm CTranslate2 làm phụ thuộc ngoài	
Qwen2.5 (1.5B)	Llama 3.2, Gemma 2, Phi-3	Cần LLM đủ nhỏ chạy CPU hoặc RAM hạn chế qua Ollama, hỗ trợ tiếng Việt tốt, giấy phép mở cho đồ án học thuật	Chính thức hỗ trợ 29 ngôn ngữ có tiếng Việt và cấp phép Apache 2.0 hoàn toàn mở cho các bản 32B (trừ 3B hoặc 72B), trong khi Llama 3.2 ràng buộc bản quyền. Ở cỡ tham số nhỏ, Qwen2.5-0.5B hoặc 1.5B hoặc 3B được báo cáo vượt Gemma2-2.6B ở nhiều tác vụ toán hoặc lập trình dù ít tham số hơn	1.5B nên vẫn hạn chế về suy luận phức tạp và độ ổn định so với model lớn hơn	[1], [16]
edge-tts	Piper	Cần giọng đọc tiếng Việt tự nhiên, chất	Tận dụng giọng neural chất lượng cao của Azure	edge-tts vẫn cần Internet tới máy chủ	[8], [17], [18]

		lượng cao mà không cần tự huấn luyện mô hình	miễn phí, không cần API key, có API bất đồng bộ tùy biến tốc độ hoặc âm lượng hoặc cao độ. Piper tuy chạy offline hoàn toàn nhưng giọng tiếng Việt hiện chủ yếu do cộng đồng tự huấn luyện (chưa chính thức, số lượng hạn chế) và bản phát triển hiện tại dùng giấy phép GPLv3. gTTS chỉ là lớp gọi không chính thức tới engine dịch của Google, tác giả tự ghi rõ "không liên kết với Google", có thể đổi API bất kỳ lúc nào	Azure việc này đi ngược một phần mục tiêu "giảm phụ thuộc cloud" của đồ á	
FastAPI	Flask, Django REST	Server cần xử lý đồng thời nhiều tác vụ I hoặc O-bound (nhận audio, gọi Ollama, gọi TTS, polling Blynk) => cần async gốc	Theo các benchmark dựa trên TechEmpower, FastAPI hoặc Uvicorn (ASGI) đạt khoảng 15.000 – 20.000 request hoặc giây, cao hơn nhiều so với Flask hoặc WSGI (khoảng 2.000 – 3.000 request hoặc giây) trên cùng phần cứng, nhờ async hoặc await gốc thay vì xử lý tuần tự, tự sinh tài liệu API	Cộng đồng, số lượng extension và tài liệu hướng dẫn của Flask vẫn lớn hơn, đường cong học async hoặc await cao hơn với người mới	[19]

			và kiểm tra kiểu dữ liệu qua Pydantic		
Blynk	Adafruit IO, Home Assistant	Cần dựng nhanh giao diện giám sát hoặc điều khiển di động không cần tự viết app, có thư viện tích hợp trực tiếp cho ESP32 hoặc ESP8266	Cung cấp trình dựng giao diện kéo-thả ngay trên app di động không cần viết code app khác Adafruit IO vốn thiên về dashboard web, khác Home Assistant vốn cần tự triển khai và duy trì hạ tầng máy chủ riêng	Gói miễn phí giới hạn số thiết bị hoặc widget và tần suất gọi API và dữ liệu vẫn đi qua cloud bên thứ ba, mâu thuẫn một phần mục tiêu "không phụ thuộc cloud"	[12], [20]
ESP32-S3 (N16R8)	ESP32 thường, ESP32-C3	Cần thu-phát âm thanh I2S đồng thời hai chiều (mic + loa), đủ RAM đệm âm thanh, có USB gốc để nạp hoặc gỡ lỗi tiện lợi	2 lõi Xtensa LX7 @240MHz, hỗ trợ PSRAM ngoài tới 16MB (bản N16R8 = 16MB Flash hoặc 8MB PSRAM Octal-SPI), có 2 bộ điều khiển I2S độc lập cho phép thu-phát đồng thời hai kênh riêng. So với ESP32 gốc (Xtensa LX6, PSRAM chỉ qua SPI chậm hơn, không USB OTG gốc) và S3 có cổng USB gốc, nạp code trực tiếp không cần mạch chuyển USB-UART rời	Giá và điện năng tiêu thụ cao hơn ESP32-C3, không hỗ trợ Bluetooth Classic, đây là điểm mạnh riêng của ESP32 gốc	[21], [22]

4.2. Hiện thực Voice Node

4.2.1. Cấu hình I2S thu âm

```

void setupMic() {
    i2s_config_t cfg = {
        .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
        .sample_rate = SAMPLE_RATE,
        .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT,
        .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
        .communication_format = I2S_COMM_FORMAT_STAND_I2S,
        .intr_alloc_flags = 0,
        .dma_buf_count = 4,
        .dma_buf_len = 1024
    };

    i2s_pin_config_t pins = {
        .bck_io_num = I2S_MIC_SCK,  #45
        .ws_io_num = I2S_MIC_WS,    #15
        .data_out_num = I2S_PIN_NO_CHANGE,
        .data_in_num = I2S_MIC_SD   #35
    };
};

```

Đoạn mã 4.1 Hàm cấu hình chân phần cứng và thông số I2S của cảm biến IMNP441. ESP32-S3 đóng vai trò I2S Master, INMP441 là Slave. Tín hiệu âm thanh từ micro được số hóa từ 32-bit thành PCM 16-bit hoặc 16kHz thông qua chuẩn I2S và truy cập bộ nhớ trực tiếp tự động chuyển dữ liệu từ I2S peripheral vào RAM mà không cần CPU can thiệp liên tục => ESP32 vẫn có thể xử lý các tác vụ khác trong lúc thu âm.

Sau 5 giây thu âm, dữ liệu PCM thô được đóng gói thành định dạng WAV và thêm header 44 byte rồi gửi qua HTTP POST tới AI Server.

4.2.2. Cấu hình I2S phát âm thanh và Ring Buffer

```

void setupSpeaker() {
    i2s_config_t cfg = {
        .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),
        .sample_rate = SAMPLE_RATE,
        .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
        .channel_format = I2S_CHANNEL_FMT_RIGHT_LEFT,
        .communication_format = I2S_COMM_FORMAT_STAND_I2S,
        .intr_alloc_flags = 0,
        .dma_buf_count = 8,
        .dma_buf_len = 512,
        .use_apll = false,
        .tx_desc_auto_clear = true,
        .fixed_mclk = 0
    };

    i2s_pin_config_t pins = {
        .bck_io_num = I2S_SPK_BCLK,  #5
        .ws_io_num = I2S_SPK_LRC,    #6
        .data_out_num = I2S_SPK_DIN,  #4
        .data_in_num = I2S_PIN_NO_CHANGE
    };
};

```

Đoạn mã 4.2 Hàm cấu hình chân phần cứng và thông số I2S của cảm biến MAX98357

```

#Cơ chế Ring Buffer và căn chỉnh dữ liệu Stream 4-byte (Stereo 16-bit) khi phát:
uint8_t chunk[1024];
int chunkFilled = 0;
const float GAIN = 3.5f; # Hệ số khuếch đại phần mềm chống rè
while ((remaining > 0 || chunkFilled > 0) && (millis() - tPlay < 120000UL)) {
    # Đọc dữ liệu từ TCP stream vào bộ đệm chunk
    if (client.available() > 0 && remaining > 0) {
        int space = sizeof(chunk) - chunkFilled;
        int toRead = min(space, (int)remaining);
        int r = client.read(chunk + chunkFilled, toRead);
        if (r > 0) {
            chunkFilled += r;
            remaining -= r;
        }
    }
    #Căn chỉnh số byte là bội số của 4 (Stereo 16-bit = 2 kênh * 2 byte = 4 byte/frame)
    int framesBytes = (chunkFilled / 4) * 4;
    if (framesBytes > 0) {
        amplifyBuffer(chunk, framesBytes, GAIN);
        size_t written = 0;
        i2s_write(I2S_SPK_PORT, chunk, framesBytes, &written, pdMS_TO_TICKS(200));

        #Đẩy các byte dư lẻ (1 đến 3 byte) về đầu bộ đệm cho chu kỳ kế tiếp
        int leftover = chunkFilled - framesBytes;
        for (int i = 0; i < leftover; i++) {
            chunk[i] = chunk[framesBytes + i];
        }
        chunkFilled = leftover;
    }
}
}

```

Đoạn mã 4.3 Vòng lặp biểu diễn cơ chế Ring Bufer

Server trả về file WAV Stereo 16kHz 16-bit ESP32-S3 nhận toàn bộ WAV qua HTTP response => bỏ qua 44 byte header => đẩy dữ liệu PCM vào bộ đệm Ring Buffer. Ring Buffer hoạt động theo mô hình sản xuất và sử dụng một task nạp dữ liệu vào buffer, một task I2S TX tự động kéo dữ liệu ra phát qua loa mà không bị ngắt quãng. MAX98357A nhận tín hiệu số I2S và tự khuếch đại thành tín hiệu analog để phát ra loa.

4.3. Hiện thực Weather Node

4.3.1. Đọc dữ liệu cảm biến BMP280

```

void setup() {
    Serial.begin(9600);
    bmpOk = bmp.begin(0x76);
    if (!bmpOk) {
        Serial.println(F("LOI: Không tìm thấy BMP280 lúc khởi động!"));
        baoLoiChoServer("Khởi động lỗi: không tìm thấy cảm biến BMP280");
    } else {
        bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                        Adafruit_BMP280::SAMPLING_X2,
                        Adafruit_BMP280::SAMPLING_X16,
                        Adafruit_BMP280::FILTER_X16,
                        Adafruit_BMP280::STANDBY_MS_500);
    }
}

void doVaGuiDuLieu() {
    float t = bmp.readTemperature();
    float p = bmp.readPressure() / 100.0F;
}

```

Đoạn mã 4.4 Hàm cấu hình chân phần cứng và thông số I2C của cảm biến BMP280

BMP280 giao tiếp với ESP8266 qua chuẩn I2C bằng hai dây SDA và SCL. Mỗi lần đọc, ESP8266 gửi địa chỉ thiết bị (0x76) + lệnh đọc lên bus I2C => BMP280 trả về giá trị nhiệt độ và áp suất. Dữ liệu được đọc 2 lần liên tiếp rồi tính trung bình để giảm nhiễu.

4.3.2. Gửi dữ liệu định kỳ lên server

```
void guiDuLieuChoServer(float t, float p) {
    if (WiFi.status() != WL_CONNECTED) return;

    WiFiClient client;
    HTTPClient http;
    String url = String("http://") + SERVER_HOST + ":" + String(SERVER_PORT) + SENSOR_UPDATE_PATH;

    http.begin(client, url);
    http.addHeader("Content-Type", "application/json");

    String json = "{\"temperature_c\":\"" + String(t, 1) + "\", \"pressure_hpa\":\"" + String(p, 1) + "\"}";
    int code = http.POST(json);

    Serial.print(F("=> Gui AI Server, ma HTTP: ")); Serial.println(code);
    http.end();
}
```

Đoạn mã 4.5 Hàm thực hiện gửi dữ liệu lên server

Vì ESP8266 không kết nối trực tiếp với Blynk nên sẽ tiến hành gửi một HTTP POST mang JSON chứa temperature_c và pressure_hpa tới AI Server. Server sẽ lưu giá trị này vào biến latest_sensor và đồng thời ghi vào sensor_data.json để không mất dữ liệu khi server khởi động lại.

4.4. Hiện thực Local AI Server

4.4.1. Module chuyển giọng nói thành văn bản

```
_whisper_model = WhisperModel("medium", device="cpu", compute_type="int8")
def transcribe_audio(wav_path: str):
    segments, info = _whisper_model.transcribe(
        wav_path, beam_size=5, language="vi"
    )
    text = " ".join(s.text.strip() for s in segments).strip()
    return text, info.language, info.language_probability
```

Đoạn mã 4.6 Hàm thực hiện chuyển từ giọng nói user sang text

Model Whisper medium được nạp một lần duy nhất khi server khởi động. compute_type="int8" Biến lượng tử hóa trọng số model từ float32 => int8 giảm RAM 4 lần và tăng tốc trên CPU khi không có GPU.

beam_size=5: Sử dụng thuật toán Beam Search duyệt 5 ứng cử viên cùng lúc để chọn ra phiên âm xác suất cao nhất.

4.4.2. Module suy luận ngôn ngữ

```
def ask_qwen(text: str, sensor: dict) -> str:
    sensor_info = ""
    ... #Cài đặt thông số
    payload = {
        "model": config.QWEN_MODEL, #qwen2.5:1.5b-instruct
        "messages": [
            {"role": "system", "content": system_content},
            {"role": "user", "content": text},
        ],
        "stream": False,
    }
    response = requests.post(config.OLLAMA_URL, json=payload, timeout=120)
    response.raise_for_status()
    return response.json()["message"]["content"]
```

Đoạn mã 4.7 Hàm thực hiện suy luận

Sử dụng kỹ thuật Context Grounding thay vì cho LLM tự bịa dữ liệu thời tiết thì server nhúng số liệu thực tế từ BMP280 vào System Prompt trước khi hỏi. Sau đó Qwen2.5 chạy cục bộ qua Ollama => toàn bộ suy luận diễn ra trên máy không gửi câu hỏi lên Internet. Kết quả trả về là câu trả lời ngắn 1-2 câu, tối ưu cho việc đọc qua TTS để phát ra loa.

4.4.3. Module tổng hợp giọng nói và truyền phát theo đoạn

```
async def text_to_speech_wav(text: str, ts: str = None) -> bytes:
    communicate = edge_tts.Communicate(
        text=text,
        voice=config.TTS_VOICE, # vi-VN-NamMinhNeural
        rate=config.TTS_RATE, # +15% (tốc độ đọc tự nhiên)
        volume=config.TTS_VOLUME, # +50%
        pitch=config.TTS_PITCH,
    )
    await communicate.save(mp3_path)
    audio = AudioSegment.from_mp3(mp3_path)
    audio = audio.normalize() + 4
    audio = audio.set_frame_rate(16000).set_channels(2).set_sample_width(2)
    pcm_data = audio.raw_data
    buf = io.BytesIO()
    with wave.open(buf, "wb") as wf:
        wf.setnchannels(2)
        wf.setsampwidth(2)
        wf.setframerate(16000)
        wf.writeframes(pcm_data)
    wav_bytes = buf.getvalue()
    return wav_bytes
```

Đoạn mã 4.8 Hàm thực hiện chuyển đổi giọng nói và phát

Edge-TTS sử dụng WebSocket kết nối tới Azure để tổng hợp giọng đọc tự nhiên chất lượng sau đó Pydub chuyển đổi định dạng hoàn toàn local => giảm sample rate từ 48kHz => 16kHz để phù hợp với cấu hình I2S của ESP32-S3 giảm băng thông truyền đi 3 lần. Header WAV 44 được tạo đúng chuẩn để ESP32 có thể bỏ qua header, đọc thẳng PCM mà không cần giải mã thêm.

4.4.4. Rate Limiter

```
_lock = threading.Lock()
_last_call = 0.0
def update(pin: str, value) -> bool:
    global _last_call
    with _lock:
        wait = config.BLYNK_MIN_INTERVAL - (time.time() - _last_call)
        if wait > 0:
            time.sleep(wait)
        try:
            ...
            resp = requests.get(..., timeout=5)
            _last_call = time.time()
            return resp.status_code == 200
        except Exception as e:
            print(f"[Blynk] LOI: {e}")
            return False
```

Đoạn mã 4.9 Hàm giới hạn tần suất gọi API

Blynk Cloud Free Plan chặn nếu gọi API > 100 lần hoặc phút nên trong quy trình file main.py gọi blynk_client.update() 4 lần liên tiếp để cập nhật trạng thái STT => LLM => TTS => lịch sử V5. Trường hợp không có Rate Limiter, Blynk sẽ trả về HTTP 429 Too Many Requests và đóng kết nối. BLYNK_MIN_INTERVAL = 0.4s => 4 lần × 0.4s = tối thiểu 1.6s dành riêng cho Blynk trong mỗi vòng lặp.

4.4.5. Emergency và Fallback Handler

```
text, lang, conf = await asyncio.to_thread(ai_pipeline.transcribe_audio, in_path)

if not text:
    await asyncio.to_thread(blynk_client.update,
        config.PIN_STATUS, f"[{now_str}] IDLE (không nghe rõ)")
    reply_wav = await ai_pipeline.text_to_speech_wav(
        "Xin lỗi, tôi không nghe rõ. Bạn nói lại được không?", ts=ts
    )
    return Response(content=reply_wav, media_type="audio/wav")
```

Đoạn mã 4.10 Lớp dự phòng khi STT thất bại

Tránh gửi chuỗi rỗng vào LLM => Qwen sẽ suy luận vô nghĩa và tốn 10–15 giây.

```
except Exception:
    error_detail = traceback.format_exc()
    print("ERR DETAIL:\n", error_detail)
    await asyncio.to_thread(blynk_client.update,
        config.PIN_STATUS, f"[{now_str}] IDLE (Lỗi AI Server)")
    return JSONResponse(status_code=500, content={"error": error_detail})
```

Đoạn mã 4.11 Lớp dự phòng cho try hoặc expect của quy trình tổng quát

Dù Whisper crash, Ollama timeout, hay TTS lỗi => ESP32 vẫn nhận được HTTP 500 response và thoát khỏi trạng thái chờ. Nếu không có tầng này, ESP32 sẽ chờ mãi.

```
def load_cached_sensor():
    if os.path.exists("sensor_data.json"):
        with open("sensor_data.json", "r") as f:
            latest_sensor.update(json.load(f))
        return

    t_resp = requests.get(f"{BLYNK_BASE}/get?token=...&V2", timeout=4)
    p_resp = requests.get(f"{BLYNK_BASE}/get?token=...&V3", timeout=4)
    if t_resp.status_code == 200 and p_resp.status_code == 200:
        latest_sensor["temperature_c"] = float(t_resp.text)
        latest_sensor["pressure_hpa"] = float(p_resp.text)
        save_cached_sensor()
```

Đoạn mã 4.12 Hàm khôi phục dữ liệu sau khi server restart

4.5. Tích hợp bảng điều khiển giám sát Blynk

```
BLYNK_WRITE(VPIN_START_STOP) {
    int val = param.asInt();
    if (val == 1) {
        if (currentState == ST_IDLE) {
            recordRequested = true;
            Serial.println("[Blynk] V1: Da chap nhan yeu cau ghi am!");
        } else {
            Blynk.virtualWrite(VPIN_START_STOP, 0);
        }
    }
}

BLYNK_WRITE(VPIN_EMERGENCY) {
    int val = param.asInt();
    if (val == 1) {
        cancelRequested = true;
        recordRequested = false;
        Serial.println("[Blynk] V6: YEU CAU HUY THAO TAC -> VE IDLE!");
        Blynk.virtualWrite(VPIN_EMERGENCY, 0);
        Blynk.virtualWrite(VPIN_START_STOP, 0);
    }
}

BLYNK_CONNECTED() {
    Serial.println("[Blynk] *** KET NOI THANH CONG *** Device: ONLINE");
    Blynk.virtualWrite(VPIN_START_STOP, 0);
    Blynk.virtualWrite(VPIN_EMERGENCY, 0);
}
```

Đoạn mã 4.13 Các hàm điều khiển hệ thống qua Blynk

Bảng 4.3 Bảng định nghĩa Virtual Pin và nguyên lý hoạt động

Virtual Pin	Loại Widget	Kiểu dữ liệu	Chức năng chi tiết
V1	Button	Integer	Kích hoạt hoặc Cho phép ghi âm Voice Node
V2	Value Display	Double	Hiển thị giá trị Nhiệt độ (°C) từ Weather Node
V3	Value Display	Double	Hiển thị giá trị Áp suất khí quyển (hPa)
V4	Terminal	String	Hiển thị trạng thái tiến trình AI
V5	Terminal	String	Nhật ký lịch sử hỏi - đáp thoại
V6	Button	Integer	Dừng khẩn cấp hoặc Hủy tác vụ ngay lập tức

4.6. Kiểm thử hệ thống

4.6.1. Kiểm thử module chuyển giọng nói thành văn bản

Cấu hình: Model Whisper medium, chạy lượng tử hóa int8 trên CPU.

Dữ liệu đầu vào: File âm thanh sample_cau_hoi_thoi_tiet.wav (Dung lượng: 127.5 KB, định dạng chuẩn PCM 16-bit, 16 kHz).

Kết quả kiểm thử cho ra output:

```
[Input] File am thanh: sample_cau_hoi_thoi_tiet.wav (127.50 KB, WAV PCM 16kHz)
[Output] Van ban nhan dang: 'Thời tiết hôm nay thế nào?'
[Output] Ngon ngu xac dinh: vi (Xac suat: 100.0%)
[Metric] Thời gian xử lý STT: 9.149 giây
[Đánh giá] Kết quả: PASS
```

Mô hình nhận dạng chính xác 100% câu hỏi tiếng Việt có dấu, không bị nuốt từ. Thời gian xử lý khoảng ~9.15s trên CPU

4.6.2. Kiểm thử module suy luận ngôn ngữ lớn

Cấu hình: Model qwen2.5:1.5b-instruct chạy cục bộ trên máy chủ qua Ollama API.

Thực nghiệm 1 (Câu hỏi danh tính thông thường không kèm dữ liệu cảm biến):

Đầu vào: "Bạn là ai?".

Đầu ra: "Tôi là một AI thông minh được thiết kế để giúp đỡ người dùng trong việc tìm kiếm thông tin và giải đáp các câu hỏi..."

Thời gian suy luận: 11.034s.

Thực nghiệm 2 (Câu hỏi thời tiết có ngữ cảnh cảm biến):

Đầu vào câu hỏi: "Thời tiết hôm nay thế nào?"

Ngữ cảnh cảm biến nạp vào prompt: Nhiệt độ = 31.0°C, Áp suất = 1039.0 hPa được đo bởi Weather Node ESP8266.

Kết quả thực nghiệm:

```
[Input 2.2 Sensor Context]: Nhiệt độ = 31.0°C, Áp suất = 1039.0 hPa
[Qwen] Sensor data khi xử lý: {'temperature_c': 31.0, 'pressure_hpa': 1039.0, 'received_at': '11:45:00 15/09/2026'}
[Qwen] Đã nạp thông tin thời tiết/nhiệt độ: 31.0°C, 1039.0hPa vào prompt.
[Output 2.2]: 'Sáng nay, nhiệt độ ngoài trời dao động trong khoảng 31.0 độ C. Áp suất khí quyển hôm nay là 1039.0 hPa.'
[Metric 2.2] Thời gian suy luận: 5.799 giây
[Grounding Check] LLM đã sử dụng trực tiếp số liệu cảm biến: CÓ
[Đánh giá] Kết quả: PASS
```

Mô hình không tự bịa đặt dữ liệu mà trích xuất và gắn kết trực tiếp số liệu 31.0°C và 1039.0 hPa vào câu trả lời tự nhiên. Thời gian phản hồi nhanh 5.799s.

4.6.3. Kiểm thử tổng hợp giọng nói

Cấu hình: Giọng đọc vi-VN-NamMinhNeural, tốc độ +15%, âm lượng +50%.

Dữ liệu đầu vào: "Nhiệt độ hiện tại là 31 độ C, áp suất khí quyển 1039 hPa.

Thời tiết hôm nay khá nắng và ấm."

Kết quả thực nghiệm:

```
[Input] Văn bản cần tổng hợp: 'Nhiệt độ hiện tại là 31 độ C, áp suất khí quyển 1039 hPa...'
[TTS] Đã lưu MP3 trả lời vào: answers\tra_loi_test_tts_unit.mp3
[TTS] Đã tạo WAV stereo 16kHz âm lượng cao (496172 byte) để truyền tới ESP32.
[Output] Tổng dung lượng: 484.54 KB (496,172 bytes)
[Output] Thời lượng âm thanh tạo ra: ~7.75 giây
[Metric] Thời gian tạo giọng nói (TTS): 5.366 giây
[Đánh giá] Kết quả: PASS
```

File âm thanh tạo ra có định dạng sóng nhị phân chuẩn header WAV 44-byte, biên độ được chuẩn hóa +4dB phát rõ nét qua loa MAX98357A mà không bị rè.

4.6.4. Kiểm thử module REST Client & Rate Limiter

Cấu hình: Giao thức HTTP REST API tới máy chủ blynk.cloud, chu kỳ tối thiểu 0.4s.

Kết quả thực nghiệm:

```
[Input] Gửi cập nhật trạng thái lên Virtual Pin V4: '[TEST] KIỂM THU ĐƠN VI BLYNK REST API'
[Blynk] update v4='[TEST] KIỂM THU ĐƠN VI BLYNK REST API' -> HTTP 200
[Metric] Thời gian phản hồi API Blynk: 0.593 giây
[Rate Limiter Test] Gọi tiếp lần 2 ngay lập tức để kiểm tra cơ chế gian cách...
[Blynk] update v4='[TEST] RATE LIMITER CHECK' -> HTTP 200
[Output] Kết quả lần 2: THÀNH CÔNG, Thời gian (có sleep gian cách): 0.987 giây
[Đánh giá] Kết quả: PASS
```

Hệ thống đảm bảo ghi nhận dữ liệu đám mây thành công HTTP 200, cơ chế Rate Limiter tự động chặn các luồng ghi liên tiếp để tránh bị server Blynk khóa tạm thời tránh được lỗi HTTP 429 Too Many Requests.

4.6.5. Kiểm thử tích hợp toàn hệ thống

Quá trình kiểm thử tích hợp mô phỏng toàn bộ luồng hoạt động khép kín giữa Weather Node (ESP8266), Voice Node (ESP32-S3) và Local AI Server.

Kịch bản 1: Hỏi đáp thời tiết với số liệu BMP280 thời gian thực theo quy trình kích hoạt sau:

Bước 1 Weather Node: Cảm biến BMP280 gửi gói JSON nhiệt độ 31°C và áp suất 1039.0 hPa lên AI Server.

Bước 2 Voice Node: Người dùng nhấn nút V1, phát âm câu hỏi: "Thời tiết hôm nay thế nào?". ESP32 ghi âm và gửi stream WAV nhị phân lên server.

Bước 3 AI Pipeline: Server chạy STT rồi đến LLM sau đó đến TTS và cuối cùng là qua Blynk Terminal.

Bước 4 Phát loa: ESP32 nhận stream âm thanh từ server và phát ra loa MAX98357A.

Kết quả thực nghiệm thu được:

```

=====
--- KICH BAN KIEM THU TICH HOP: KICH BAN 1: HOI THOI TIET (CO DU LIEU SENSOR BMP280: 31°C, 1039 hPa) ---
=====
[Buoc 1] Du lieu cam bien moi truong (Weather Node - ESP8266):
- Trang thai ket noi : DA KET NOI & CAP NHAT (BMP280 I2C)
- Nhiet do moi truong : 31.0 do C
- Ap suat khi quyen : 1039.0 hPa
- Thoi diem dong bo : 23:57:19 15/09/2026

[Buoc 2] Du lieu am thanh thu nhan (Voice Node - ESP32-S3):
- File am thanh : e2e_cau_hoi_thoi_tiet.wav
- Dung luong audio : 126.04 KB (PCM WAV 16kHz Stereo)
- Cau hoi mo phong : "Thời tiết hôm nay thế nào?"

[Buoc 3] Thuc thi Pipeline AI Server (End-to-End):
- [3.1 STT - Whisper] : "Thời tiết hôm nay thế nào?" (Do tre: 10.318s | Ngon ngu: vi | Do tin cay: 100.0%)
- [3.2 LLM - Qwen2.5] : "Thời tiết hôm nay là một ngày mưa nhẹ. Nhiệt độ hiện tại là 31.0 độ C và áp suất khí quyển là 1039.0 hPa." (Do tre: 11.976s)
- [3.3 TTS - EdgeTTS] : Da tong hop am thanh WAV (Dung luong: 534.04 KB | Do tre: 9.671s)
- [3.4 IoT - Blynk ] : Dong bo Terminal V5 (Trang thai: OK 200 | Do tre: 0.616s)

[Buoc 4] Danh gia ket qua kiem thu & Phan ra do tre:
- Tong thoi gian xu ly (E2E Latency) : 32.582 giay
- Phan ra do tre tung thanh phan:
  + Nhan dang giong noi (STT) : 10.318s (31.7%)
  + Suy luan ngon ngu (LLM) : 11.976s (36.8%)
  + Tong hop giong noi (TTS) : 9.671s (29.7%)
  + Dong bo IoT Cloud (Blynk) : 0.616s (1.9%)
- Am thanh tra ve cho ESP32 phat : 534.04 KB (Stereo 16-bit 16kHz PCM)
- Tieu chi kiem dinh noi dung : DAT (Grounding chinh xac thong so 31 do C, 1039 hPa)
- Ket luan kiem thu kich ban : PASS (He thong dap ung dung thiet ke)
=====

```

Kịch bản 2: Hỏi "Bạn là ai?" theo quy trình kích tương tự cho ra kết quả thực nghiệm sau:

```

=====
--- KICH BAN KIEM THU TICH HOP: KICH BAN 2: HOI DANH TINH TRO LY LEVA (KHONG CO DU LIEU SENSOR) ---
=====
[Buoc 1] Du lieu cam bien moi truong (Weather Node - ESP8266):
- Trang thai ket noi : KHONG CO DU LIEU (CHUA KET NOI SENSOR)
- Nhiet do moi truong : N/A
- Ap suat khi quyen : N/A
- Thoi diem dong bo : N/A

[Buoc 2] Du lieu am thanh thu nhan (Voice Node - ESP32-S3):
- File am thanh : e2e_cau_hoi_danh_tinh.wav
- Dung luong audio : 117.04 KB (PCM WAV 16kHz Stereo)
- Cau hoi mo phong : "Bạn là ai?"

[Buoc 3] Thuc thi Pipeline AI Server (End-to-End):
- [3.1 STT - Whisper] : "Bạn là ai?" (Do tre: 8.552s | Ngon ngu: vi | Do tin cay: 100.0%)
- [3.2 LLM - Qwen2.5] : "Tôi là một chương trình trí tuệ nhân tạo (AI) được tạo ra bởi các chuyên gia trong lĩnh vực công nghệ thông tin. Tôi được thiết kế để đáp ứng yêu cầu và câu hỏi của người dùng bằng cách cung cấp thông tin chính xác, cập nhật và có tính khoa học cao. Tôi có thể giúp bạn với nhiều loại bài tập và các vấn đề khác nhau." (Do tre: 5.422s)
- [3.3 TTS - EdgeTTS] : Da tong hop am thanh WAV (Dung luong: 979.54 KB | Do tre: 8.988s)
- [3.4 IoT - Blynk ] : Dong bo Terminal V5 (Trang thai: OK 200 | Do tre: 0.592s)

[Buoc 4] Danh gia ket qua kiem thu & Phan ra do tre:
- Tong thoi gian xu ly (E2E Latency) : 23.554 giay
- Phan ra do tre tung thanh phan:
  + Nhan dang giong noi (STT) : 8.552s (36.3%)
  + Suy luan ngon ngu (LLM) : 5.422s (23.0%)
  + Tong hop giong noi (TTS) : 8.988s (38.2%)
  + Dong bo IoT Cloud (Blynk) : 0.592s (2.5%)
- Am thanh tra ve cho ESP32 phat : 979.54 KB (Stereo 16-bit 16kHz PCM)
- Tieu chi kiem dinh noi dung : DAT (LLM tu luan giai hop ly)
- Ket luan kiem thu kich ban : PASS (He thong dap ung dung thiet ke)
=====

```

Với cả hai kịch bản hệ thống sau khi hoàn thành bốn bước cho ra độ trễ thực hiện tại từng module cụ thể. Đồng thời có thể nhận biết được khi truy xuất dữ liệu từ cảm biến AI sẽ tốn nhiều thời gian để suy luận hơn so với câu hỏi không cần truy xuất dữ liệu.

4.6.6. Bảng tra kiểm thử và kết quả

Bảng 4.4 Bảng đánh giá tổng quan hệ thống sau kiểm thử

Tiêu chí so sánh	Kịch bản 1 (E2E-01): Hỏi thời tiết	Kịch bản 2 (E2E-02): Hỏi danh tính
Dữ liệu cảm biến đầu vào	31.0 °C & 1039.0 hPa	N hoặc A (Chưa kết nối)
Câu hỏi âm thanh đầu vào	"Thời tiết hôm nay thế nào?"	"Bạn là ai?"
Dung lượng Audio đầu vào	126.04 KB	117.04 KB
Độ trễ STT (Faster-Whisper)	10.318 s (31.7%)	8.552 s (36.3%)
Độ trễ suy luận LLM (Qwen2.5)	11.976 s (36.8%)	5.422 s (23.0%)
Độ trễ TTS (Edge-TTS)	9.671 s (29.7%)	8.988 s (38.2%)
Độ trễ Blynk Cloud	0.616 s (1.9%)	0.592 s (2.5%)
Tổng độ trễ phản hồi (E2E)	32.582 s	23.554 s
Kích thước Audio phản hồi	534.04 KB	979.54 KB
Tiêu chí kiểm định	ĐẠT (Đưa chính xác số liệu vào câu trả lời)	ĐẠT (Tự suy luận danh tính, không nhầm lẫn số liệu)
Nhận xét	ĐẠT	ĐẠT

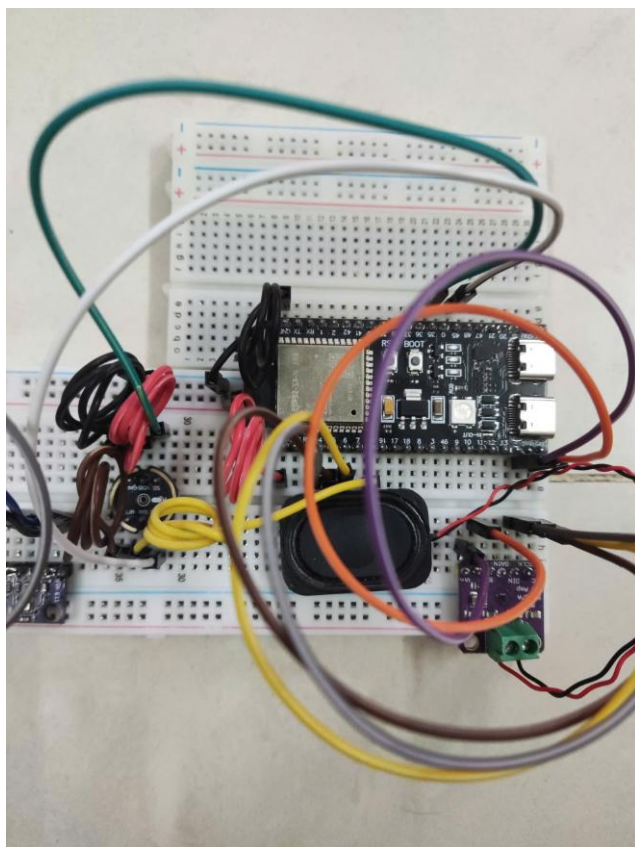
Chương 5. KẾT QUẢ, ĐÁNH GIÁ VÀ KẾT LUẬN

5.1. Kết quả thực hiện

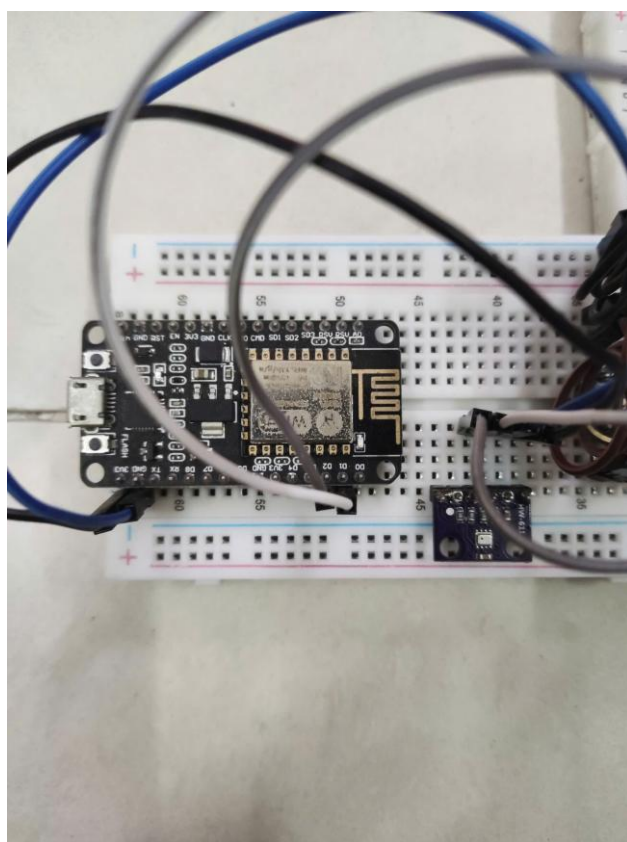
Bảng 5.1 Bảng so sánh đồ án sau thực hiện so với mục tiêu đề ra ban đầu

Mục tiêu	Mức độ đạt được	Ghi chú
Thu-phát âm thanh thời gian thực qua I2S	Đạt được một phần	Thành công thu và phát âm rõ tiếng, không bị rè. Nhưng chưa thật sự tối ưu
Quy trình AI cục bộ	Đạt	Hệ thống chạy đúng tuần tự, đưa ra câu trả lời chính xác liên quan đến câu hỏi
Độ trễ phản hồi trong ngưỡng thời gian thực	Đạt được một phần	Suy luận ngôn ngữ và tổng hợp giọng nói được nhận xét khá tốt nhưng về nhận diện giọng nói việc chạy mô hình medium có thể vẫn chưa phải tối ưu nhất dẫn đến độ trễ phản hồi hiện chưa đạt mức thời gian thực
Tích hợp dữ liệu cảm biến môi trường	Đạt	Thành công thu thập, ghi và gửi dữ liệu lên server
Giám sát và điều khiển từ xa qua IoT	Đạt	Có thể điều khiển được hệ thống từ bên ngoài

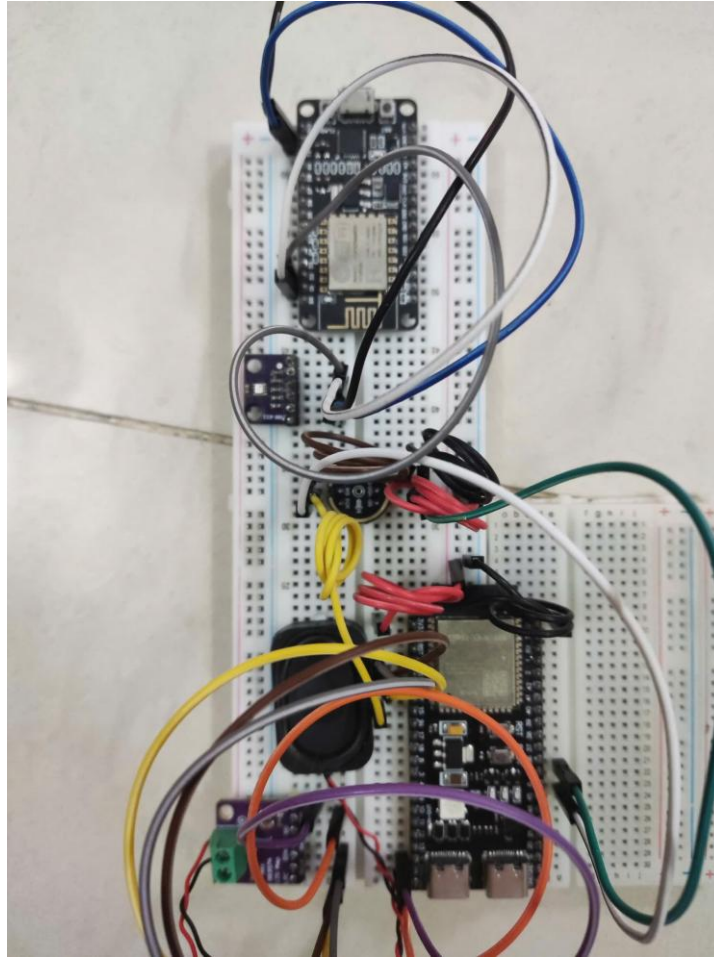
5.2. Sản phẩm



Hình 5.1 Hình ảnh sản phẩm thực tế của Voice Node (esp32s3n16r8)



Hình 5.2 Hình ảnh sản phẩm thực tế của Weather Node (esp8266)



Hình 5.3 Hình ảnh sản phẩm thực tế của toàn bộ hệ thống

5.3. Đánh giá hiệu năng hệ thống

5.3.1. Độ trễ phản hồi từng giai đoạn

Bảng 5.2 Bảng nhận xét độ trễ của chu trình tích hợp

Thành phần xử lý	Công nghệ / Giao thức	Đánh giá độ trễ
Nhận dạng giọng nói	Faster-Whisper	Lớn nhất: Do chạy mô hình medium.
Suy luận ngữ nghĩa	Qwen 2.5:1.5B	Khá tốt: Tốc độ sinh token nhanh, phản hồi ngắn gọn.
Tổng hợp âm thanh	Edge-TTS + Pydub Resample	Bình thường: Phụ thuộc đường truyền Internet tới máy chủ TTS.
Đồng bộ đám mây	HTTPS REST	Bình thường: Chạy bất đồng bộ, không làm chậm hệ thống.

5.3.1. Độ chính xác nhận diện giọng nói

Việc sử dụng Faster-Whisper với mô hình medium giúp hệ thống có khả năng nhận diện giọng nói tốt đã được kiểm thử ở trên. Nhưng bên cạnh đó việc đề án đang muốn vận hành một hệ thống thật sự tối ưu phù hợp để sử dụng trên phần cứng cá nhân của

người dùng, mô hình hình medium hiện tại cho ra kết quả có thể gọi là tốt nhất về mặt chất lượng nhưng về xét về mục tiêu thì chưa thật sự đáp ứng.

5.3.2. Độ chính xác phân tích ngữ cảnh

Hệ thống khi nhận được dữ liệu sau trạng thái phân tích giọng nói mô hình AI có thể hiểu được người dùng đang hỏi gì, cần biết dữ liệu gì và cần trả lời như thế nào cho mạch lạc, đúng nội dung người dùng hỏi bằng ngôn ngữ tiếng Việt, khi được hỏi về các nhiệm vụ cơ bản có thể trả lời được như: Nêu ngẫu nhiên 5 cái tên, kể một câu truyện, v.v. Đối với các nội dung câu hỏi cần biết dữ liệu về thời tiết như nhiệt độ hoặc áp suất không khí, AI sẽ truy xuất dữ liệu từ local server để trả lời mà không bịa đặt dữ liệu.

5.4. Đánh giá quá trình thực hiện

5.4.1. Phân tích nguyên nhân khi hệ thống sai

Các lỗi hệ thống được ghi nhận lại trong quá trình phát triển hệ thống như sau:

_ Sự ảnh hưởng đầu tiên liên quan đến tạp âm môi trường và hiện tượng hồi tiếp âm thanh Cảm biến INMP441 có độ nhạy rất cao nên khi người dùng ra lệnh trong môi trường có tiếng quạt gió, tiếng ồn giao thông hoặc tiếng tivi phát nền, mô hình Whisper dễ nhận dạng nhầm hoặc tự động chèn thêm các ký tự lạ (như "cảm ơn bạn", "hãy đăng ký kênh" do bias trong dữ liệu huấn luyện của Whisper khi gặp khoảng lặng có tiếng ồn). Đồng thời mô hình Faster-Whisper bản medium cần từ 9s – 15s để giải mã một đoạn âm thanh 5s. Nếu người dùng ngắt lời hoặc bấm nút hủy giữa chừng thì luồng STT trên server vẫn phải xử lý hết gói tin trước khi giải phóng bộ nhớ, gây ra độ trễ tích lũy cho lần nhấn tiếp theo.

_ Hay hiện tượng ảo giác tri thức khi thiếu dữ liệu đầu vào với trường hợp điển hình như node cảm biến ESP8266 bị mất kết nối hoặc server vừa khởi động lại mà chưa nhận được bản tin hoặc sensor-update, từ điển dữ liệu cảm biến bị rỗng. Nếu prompt không khóa chặt hành vi yêu cầu AI chỉ cung cấp dữ liệu thực tế, LLM có xu hướng tự suy đoán ra một mức nhiệt độ giả định ví dụ: tự trả lời "Thời tiết hôm nay khoảng 25 độ C"), gây sai lệch nghiêm trọng so với thực tế.

_ Sự giới hạn về độ dài ngữ cảnh và kích thước mô hình 1.5B tham số tối ưu hóa cho CPU, khả năng suy luận logic phức tạp bị hạn chế so với các mô hình 7B hay 70B. Nếu người dùng đặt câu hỏi quá dài giả dụ như trên 3 vế câu, mô hình sẽ bỏ qua mệnh đề đầu và chỉ trả lời mệnh đề cuối cùng.

_ Lỗi về dữ liệu âm thanh việc phát qua loa là định dạng Stereo 16-bit, tương ứng mỗi khung mẫu chiếm đúng 4 byte gồm 2 byte Left + 2 byte Right. Ban đầu, khi nhận từng khối TCP chunk có kích thước bất kỳ, nếu số byte ghi vào DMA không phải bội số của 4, ranh giới byte của kênh Trái và Phải bị đảo lộn, biến tín hiệu âm thanh thành tiếng nổ lách tách hoặc rè vỡ hoàn toàn.

_ Nguồn cung cho mạch khuếch đại MAX98357A khi đánh loa 3W 4Ω ở công suất đỉnh có thể hút dòng đột biến lên đến 800mA. Khi sử dụng chung nguồn 5V cấp từ cổng USB của máy tính mà không có tụ bù dung lượng lớn, vi điều khiển ESP32-S3 bị sụt áp dưới ngưỡng 2.8V, kích hoạt mạch ngắt làm vi điều khiển khởi động lại giữa chừng.

_ Cuối cùng là xung đột tần suất gọi API Blynk Cloud với lỗi HTTP 429 Too Many Requests vì máy chủ Blynk 2.0 Cloud áp dụng chính sách giới hạn tần suất nghiêm ngặt đối với tài khoản cá nhân. Trong các phiên bản thử nghiệm, server liên tục gọi API cập nhật trạng thái tiến trình STT đến LLM rồi tới TTS trong thời gian vài mili-giây, dẫn đến việc IP bị server Blynk tạm khóa. Liên quan đến việc chính sách giới hạn tần suất nghiêm ngặt đối với tài khoản cá nhân Widget Terminal cập nhật trên Blynk giới hạn chuỗi dữ liệu tối đa 1024 ký tự. Sau 4 - 5 câu hỏi thoại dung lượng chuỗi vượt quá 1024 ký tự khiến server Blynk phản hồi mã lỗi HTTP 400 Bad Request làm hệ thống mất khả năng cập nhật lịch sử.

5.4.2. So sánh với các giải pháp đã khảo

Bảng 5.3 Bảng so sánh giữa giải pháp của đề tài và các giải pháp khác

Tiêu chí đánh giá	Giải pháp của Đề tài	Giải pháp Cloud hoàn toàn	Giải pháp TinyML thuần túy
Nơi xử lý AI	Cục bộ gần 100% tại máy chủ LAN	Đám mây của bên thứ ba	Trực tiếp trên bộ nhớ SRAM của chip ESP32

Bảo mật & Quyền riêng tư	Dữ liệu giọng nói gốc của người dùng không rời khỏi LAN mà chỉ có văn bản phản hồi ngắn khoảng 1–2 câu được gửi qua dịch vụ TTS bên thứ ba là Microsoft Edge để tổng hợp giọng đọc.	Nguy cơ cao vì dữ liệu giọng nói được lưu trữ và phân tích trên server bên ngoài mạng LAN	Rất an toàn vì không truyền dữ liệu ra ngoài
Chi phí vận hành định kỳ	Không tốn kém vì tận dụng máy tính sẵn có, mô hình mã nguồn mở	Rất tốn kém vì tính phí theo số lượng ký tự token và phút audio AP	Không tốn chi phí máy chủ
Khả năng hiểu ngôn ngữ tự nhiên	Có thể đối thoại tự do, trả lời linh hoạt, hiểu ngữ cảnh thời tiết	Có thể xử lý đa ngữ cảnh, suy luận thông minh bậc nhất	Chỉ có thể nhận diện được từ 10 đến 20 từ khóa cố định
Độ phụ thuộc Internet	Mất Internet vẫn nhận diện giọng nói và trả lời thời tiết	Mất mạng hệ thống sẽ ngừng hoạt động	Hoạt động độc lập hoàn toàn
Xử lý ngữ cảnh phản cứng	Tự động nạp số liệu nhiệt độ từ cảm biến	Phải xây dựng hạ tầng cơ sở dữ liệu	Không đủ dung lượng RAM để ghép nối ngữ cảnh
Giao diện giám sát người dùng	Bảng điều khiển trực quan Blynk IoT 2.0	Phụ thuộc ứng dụng riêng của bên thứ ba	Thường chỉ có đèn LED hoặc còi chip đơn giản

5.5. Kết luận

5.5.1. Hạn chế của đề tài

_ Hệ thống hiện tại vẫn cho ra kết quả đáp lại từ AI khá chậm so với mục tiêu thời gian thực của đề án đề ra ban đầu.

_ Bảng điều khiển giám sát vẫn phụ thuộc vào nền tảng đám mây Blynk nên một phần dữ liệu trạng thái và lịch sử hội thoại vẫn được truyền qua hạ tầng bên thứ ba, chưa hoàn toàn đạt được mục tiêu “không phụ thuộc Cloud” một cách tuyệt đối.

_ Hệ thống chưa có khả năng phát hiện từ khóa đánh thức việc kích hoạt phụ thuộc vào thao tác thủ công trên ứng dụng di động gây bất tiện cho người dùng.

_ Thời lượng ghi âm cố định 5 giây có thể không phù hợp với mọi độ dài câu hỏi và không phù hợp với thực tế vì nhiều trường hợp khi hỏi con người có xu hướng vừa hỏi vừa suy nghĩ nên 5 giây ghi âm tạo cảm giác bị thúc giục, không thoải mái dẫn đến trường hợp câu hỏi không được mạch lạc kết quả AI đưa ra câu trả lời sai với yêu cầu trong câu hỏi.

5.5.2. Kế hoạch phát triển hệ thống trong tương lai nếu có thêm 3 tháng

Đối với hệ thống hiện tại đồ án đã xây dựng được kiến trúc phân tầng, giao tiếp I2S phần cứng, quy trình AI khép kín và tích hợp đám mây Blynk. Tuy nhiên, trong quá trình vận hành thực tế với đồ án đã xảy ra khá nhiều bất cập nên việc phát triển và tối ưu hệ thống dự kiến sẽ phát triển sau được chia làm 3 việc chính:

_ Đầu tiên là tích hợp mô hình phát hiện từ khóa đánh thức chạy trực tiếp trên vi điều khiển. Ở bối cảnh hiện tại, người dùng bắt buộc phải mở ứng dụng điện thoại Blynk và tìm đến widget và nhấn nút V1 theo cách thủ công mới bắt đầu được ghi âm. Đây là rào cản tương tác trực quan nhất làm mất đi bản chất "hands-free" của một hệ thống trợ lý giọng nói thông minh hiện đại ngày nay. Giải pháp được đề xuất hiện tại sẽ phát triển thêm một hàm có khả năng nhận biết giọng nói và tạp âm thời gian thực khi nghe có tín hiệu wake word từ người dùng, máy sẽ chào và tiến hành ghi âm. Các kỳ vọng được đặt ra như: loại bỏ hoàn toàn thao tác nhấn nút V1 trên Blynk, thời gian từ lúc người dùng nói từ khóa đến khi bắt đầu phản hồi và ghi âm câu hỏi nhanh hơn so với hiện tại và độ tiêu thụ điện năng bổ sung cho tác vụ luôn mở mic chỉ cần phải nhỏ.

_ Thứ hai là bổ sung thuật toán Phát hiện Hoạt động Giọng nói ghi âm thích nghi theo độ dài câu nói thực tế. Hiện tại hệ thống sử dụng thời lượng ghi âm cố định là 5 giây bất kể người dùng nói câu ngắn 3s. Điều này tạo ra hai vấn đề nghiêm trọng:

+ Vấn đề đầu tiên, đối với câu hỏi ngắn người dùng hỏi "Mấy giờ rồi?" hệ thống vẫn ghi âm 5s và gửi lên server 160KB dữ liệu trong đó có 3s khoảng lặng trống. Mô hình Whisper mất thêm thời gian giải mã phần âm thanh trống này, làm tăng độ trễ lên khoảng 2s không cần thiết.

+ Đối nghịch với câu hỏi dài, nếu người dùng muốn đặt câu hỏi phức tạp hơn 5s ví dụ: "Hôm nay áp suất như thế này thì theo dự báo thời tiết buổi chiều có khả năng

mưa không?" phần cuối câu sẽ bị cắt cụt khiến mô hình STT chỉ nhận được phần đầu dẫn đến câu trả lời không đúng trọng tâm.

_ Cuối cùng là xây dựng bảng điều khiển giám sát cục bộ tự lưu trữ để loại bỏ phụ thuộc vào nền tảng đám mây bên thứ ba mặc dù Blynk IoT 2.0 cung cấp giao diện giám sát trực quan và tiện lợi trên điện thoại nhưng hệ thống hiện tại vẫn đang phụ thuộc hoàn toàn vào một nền tảng đám mây của bên thứ ba nên có các bất cập về giới hạn như: giới hạn Datastream và tần suất cập nhật do tài khoản Blynk hiện tại là miễn phí chỉ hỗ trợ tối đa 30s giữa hai lần cập nhật, trong khi dữ liệu giám sát nhiệt độ thời gian thực lý tưởng cần cập nhật mỗi 5s hoặc thậm chí theo sự kiện và giới hạn lịch sử dữ liệu chỉ lưu log 2 giờ không thể phân tích xu hướng nhiệt độ và áp suất dài hạn để phục vụ dự đoán thời tiết vi mô.

TÀI LIỆU THAM KHẢO

- [1] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.
- [2] NXP Semiconductors, "UM10204: I2C-bus specification and user manual," Rev. 7.0, Oct. 2021.
- [3] J. Robert, "pydub: Manipulate audio with a simple and easy high level interface," GitHub repository, n.d.
- [4] InvenSense, “INMP441 omnidirectional microphone with bottom port and I2S digital output,” Datasheet, n.d.
- [5] Analog Devices, “MAX98357A hoặc MAX98357B PCM Class D amplifier,” Datasheet, Rev. 16, 2019.
- [6] A. Radford et al., “Robust speech recognition via large-scale weak supervision,” arXiv preprint arXiv:2212.04356, 2022.
- [7] SYSTRAN, “faster-whisper: Reimplementation of OpenAI's Whisper using CTranslate2,” GitHub repository, n.d.
- [8] rany2, "edge-tts: Use Microsoft Edge's online text-to-speech service from Python," GitHub repository.n.d.
- [9] Bosch Sensortec, “BMP280 digital pressure sensor,” Datasheet, Document Rev. 1.14, n.d.
- [10] Ollama, “Ollama documentation,”n.d.
- [11] R. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” RFC 9110, Internet Engineering Task Force, Jun. 2022.
- [12] Blynk, “Blynk documentation.”, n.d.
- [13] P. Cheng, I. E. Bagci, J. Yan, and U. Roedig, “Smart Speaker Privacy Control – Acoustic Tagging for Personal Voice Assistants,” 2019 IEEE Symposium on Security and Privacy Workshops (SPW), 2019, pp. 144–149, doi: 10.1109 hoặc SPW.2019.00035.
- [14] “Exploiting Privacy-preserving Voice Query in Healthcare-based Voice Assistant System,” 2020 IEEE Conference, IEEE Xplore, doi: 10.1109 hoặc ICC40277.2020.9149065.
- [15] ggml-org, "Further improve the speed of Whisper.cpp (might need a dependency)," GitHub Discussion #589, whisper.cpp repository, 2023.

- [16] Qwen Team (Alibaba Cloud), "Qwen2.5: A Party of Foundation Models," Qwen Blog, Sep. 2024.
- [17] OHF-Voice (formerly Rhasspy), "piper1-gpl: Fast and local neural text-to-speech engine," GitHub repository, n.d.
- [18] P. N. Durette, "gTTS: Python library and CLI tool to interface with Google Translate's text-to-speech API," GitHub repository, n.d.
- [19] Netguru, "Flask vs FastAPI: Python Framework Comparison 2026," Netguru Blog, 2026.
- [20] R. Faludi, "IoT Platforms for Makers," faludi.com, May 2025.
- [21] JLCPCB, "ESP32-C3 vs ESP32-S3: Specs, Price & Which One to Pick," JLCPCB Blog, 2026.
- [22] Wavtron, "ESP32 vs ESP32-S3 vs ESP32-C3: Which Board Should You Pick?" Wavtron Blog, Apr. 2026.

PHỤ LỤC

CÁC ĐOẠN CODE TIÊU BIỂU

Đoạn mã 4.1 Hàm cấu hình chân phần cứng và thông số I2S của cảm biến	
IMNP441.....	33
Đoạn mã 4.2 Hàm cấu hình chân phần cứng và thông số I2S của cảm biến	
MAX98357.....	33
Đoạn mã 4.3 Vòng lặp biểu diễn cơ chế Ring Bufer.....	34
Đoạn mã 4.4 Hàm cấu hình chân phần cứng và thông số I2C của cảm biến	
BMP280	34
Đoạn mã 4.5 Hàm thực hiện gửi dữ liệu lên server	35
Đoạn mã 4.6 Hàm thực hiện chuyển từ giọng nói user sang text.....	35
Đoạn mã 4.7 Hàm thực hiện suy luận.....	36
Đoạn mã 4.8 Hàm thực hiện chuyển đổi giọng nói và phát	36
Đoạn mã 4.9 Hàm giới hạn tần suất gọi API.....	37
Đoạn mã 4.10 Lớp dự phòng khi STT thất bại.....	37
Đoạn mã 4.11 Lớp dự phòng cho try hoặc expect của quy trình tổng quát.....	37
Đoạn mã 4.12 Hàm khôi phục dữ liệu sau khi server restart.....	38
Đoạn mã 4.13 Các hàm điều khiển hệ thống qua Blynk	38